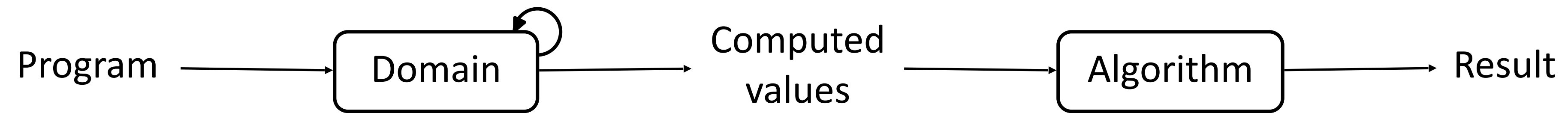


Static analysis and software verification

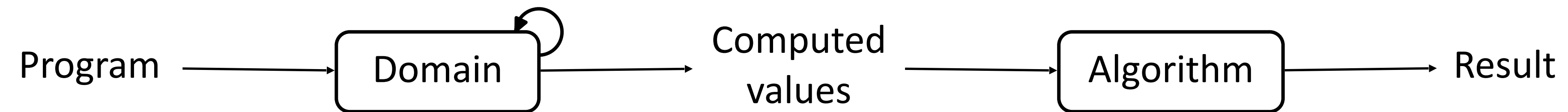
Lecture 5 - Introduction to LiSA

Vincenzo Arceri - University of Parma - vincenzo.arceri@unipr.it

Static analysis at a glance

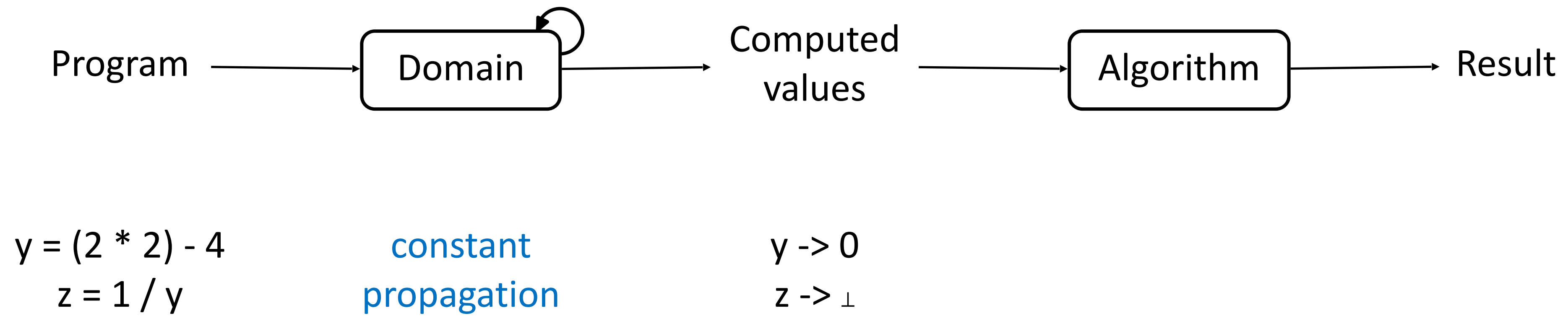


Static analysis at a glance

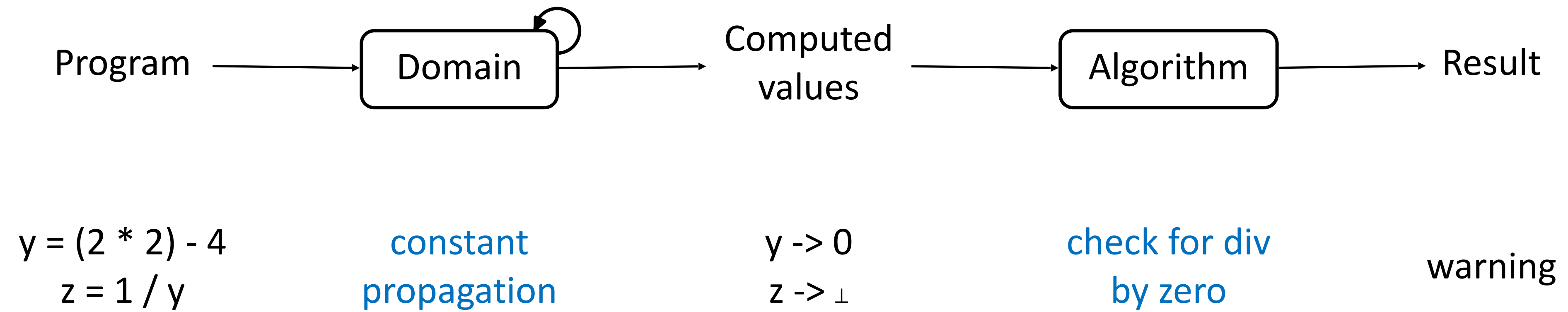


$y = (2 * 2) - 4$
 $z = 1 / y$

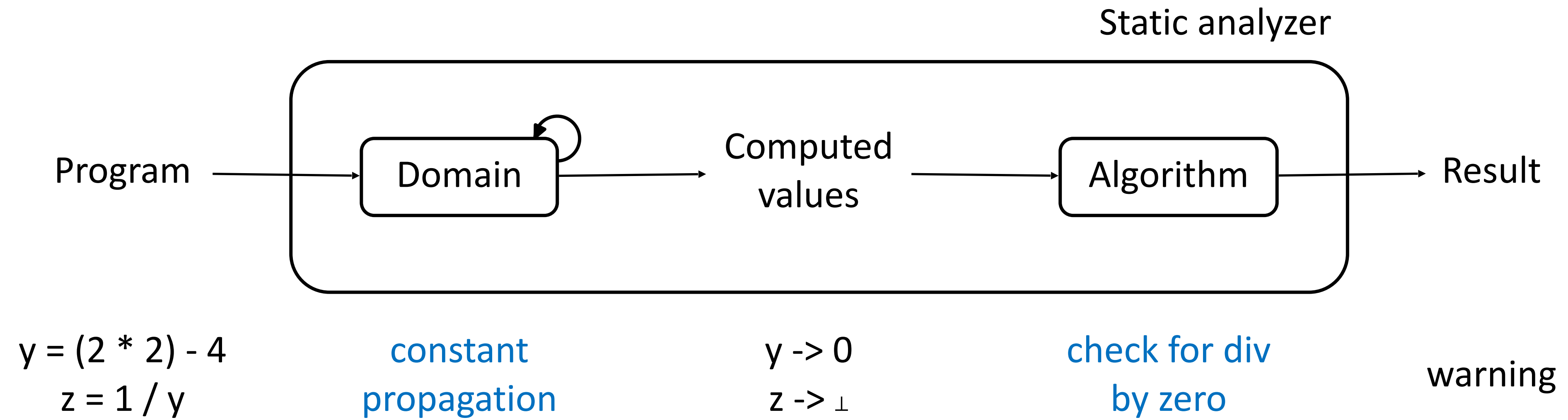
Static analysis at a glance



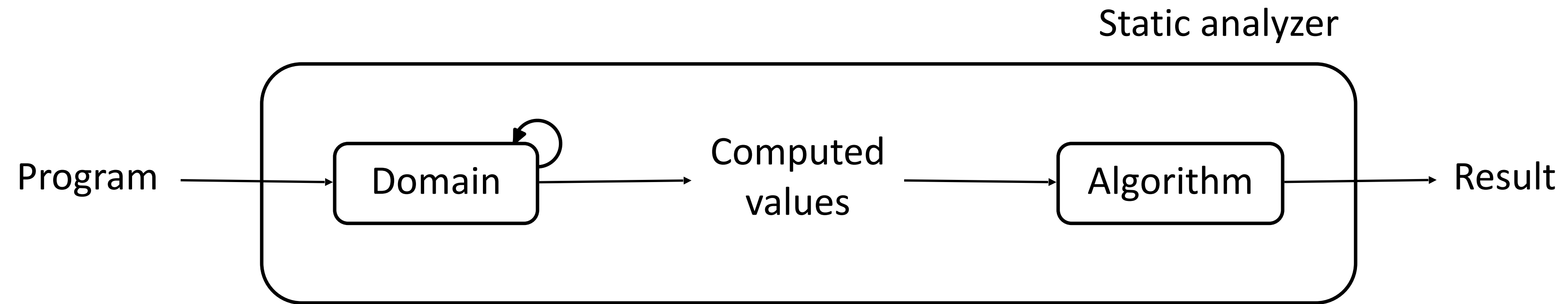
Static analysis at a glance



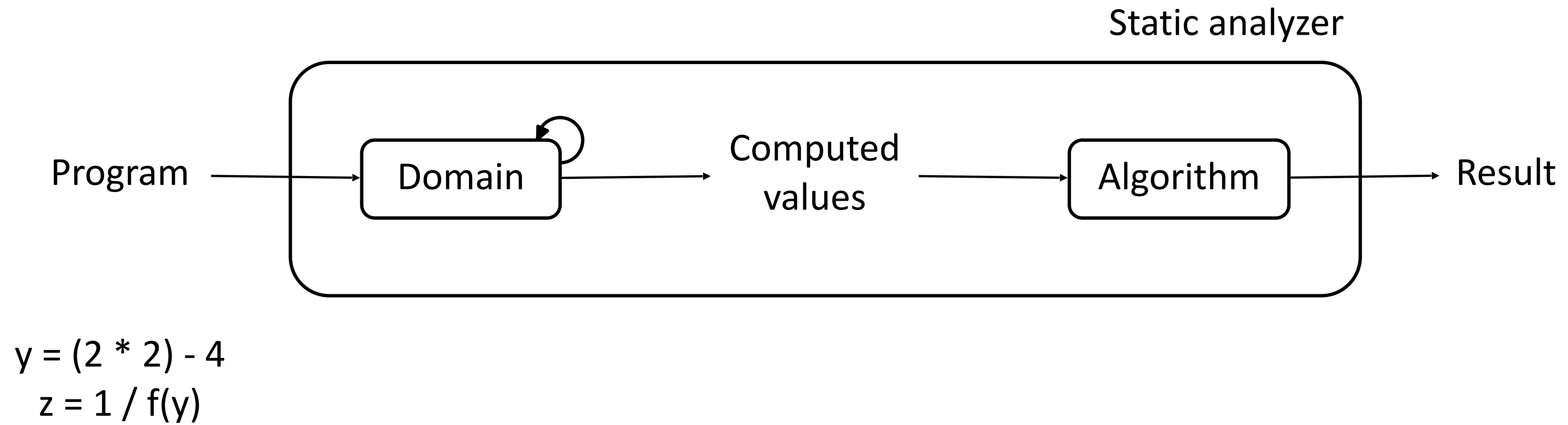
Static analysis at a glance



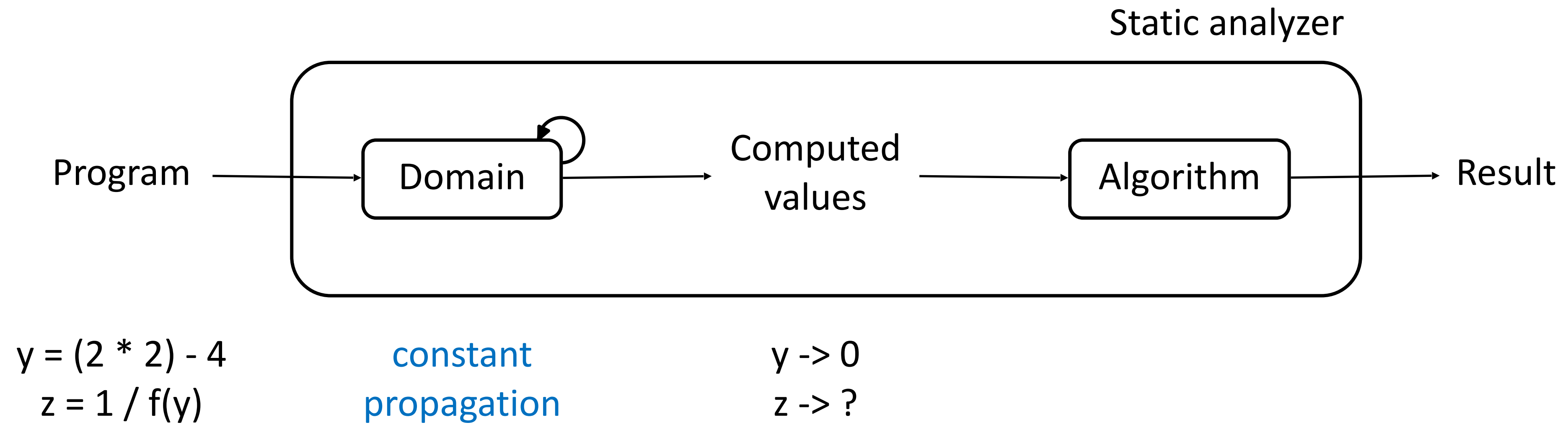
Static analysis at a glance



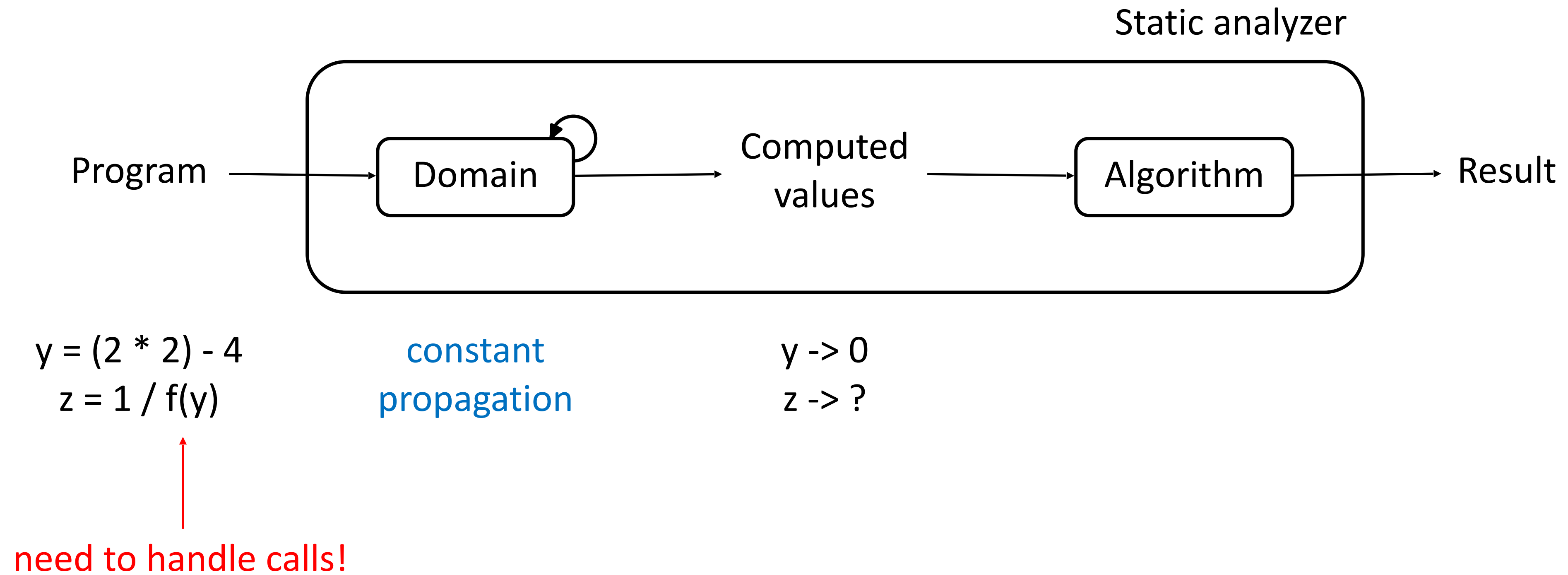
Static analysis at a glance



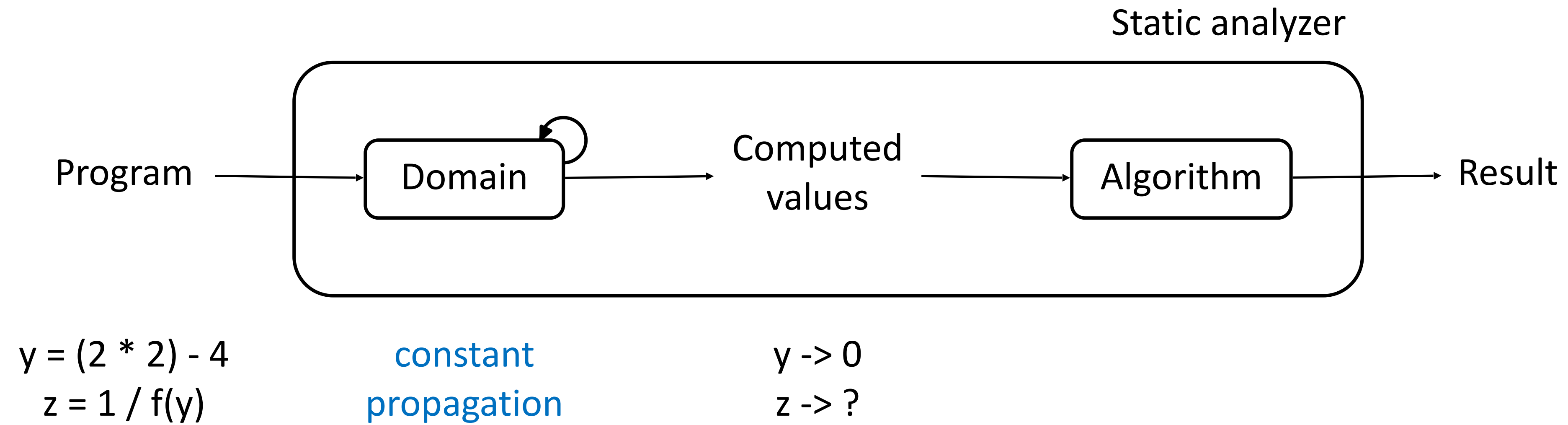
Static analysis at a glance



Static analysis at a glance

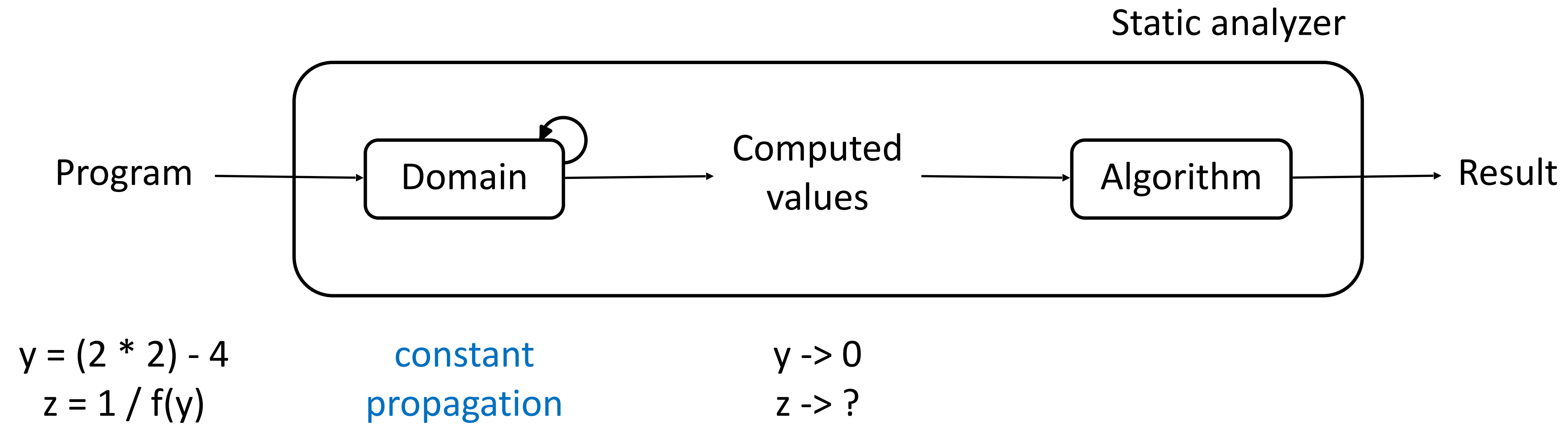


Static analysis at a glance



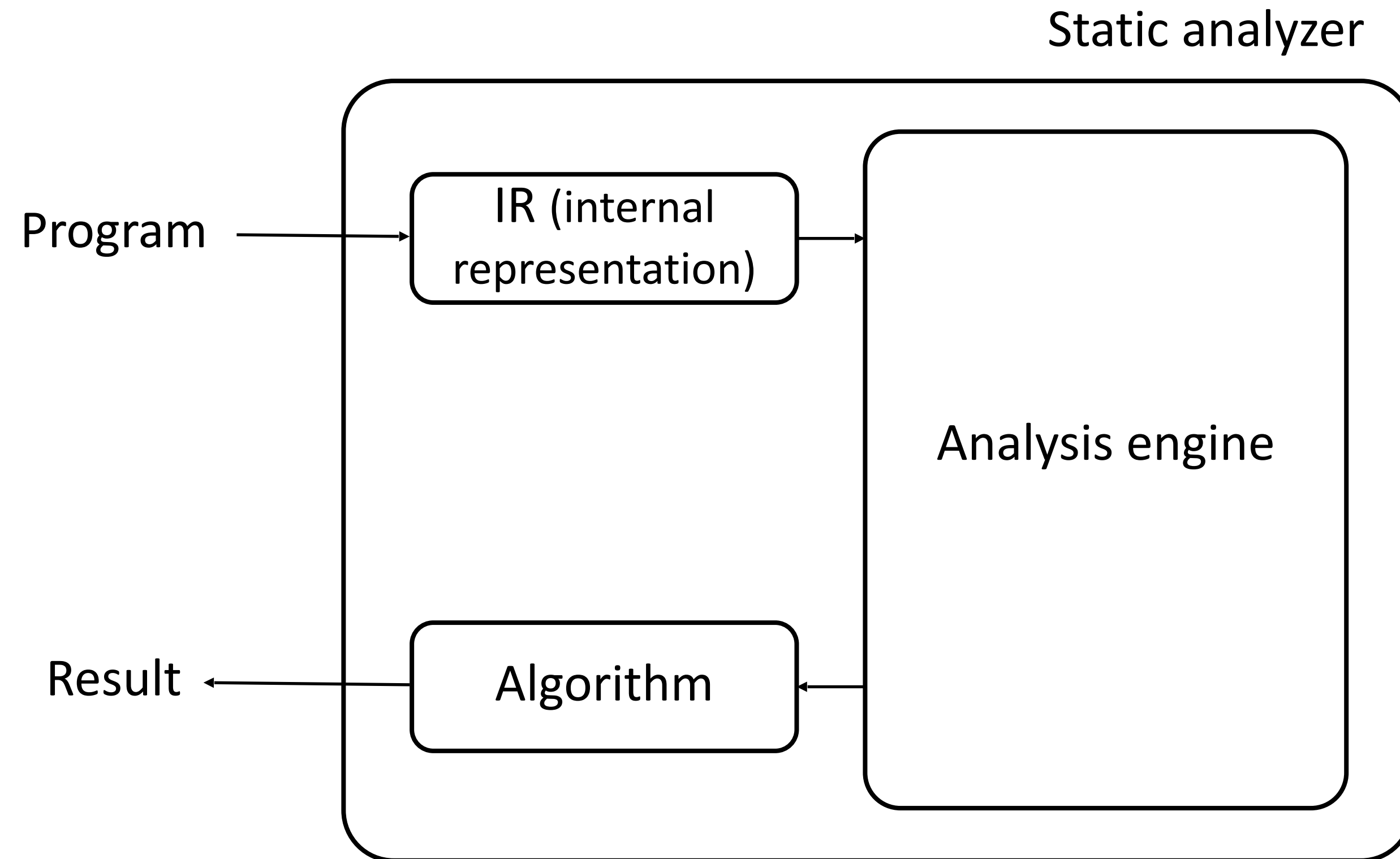
need to handle calls!
but what if we change domain?

Static analysis at a glance

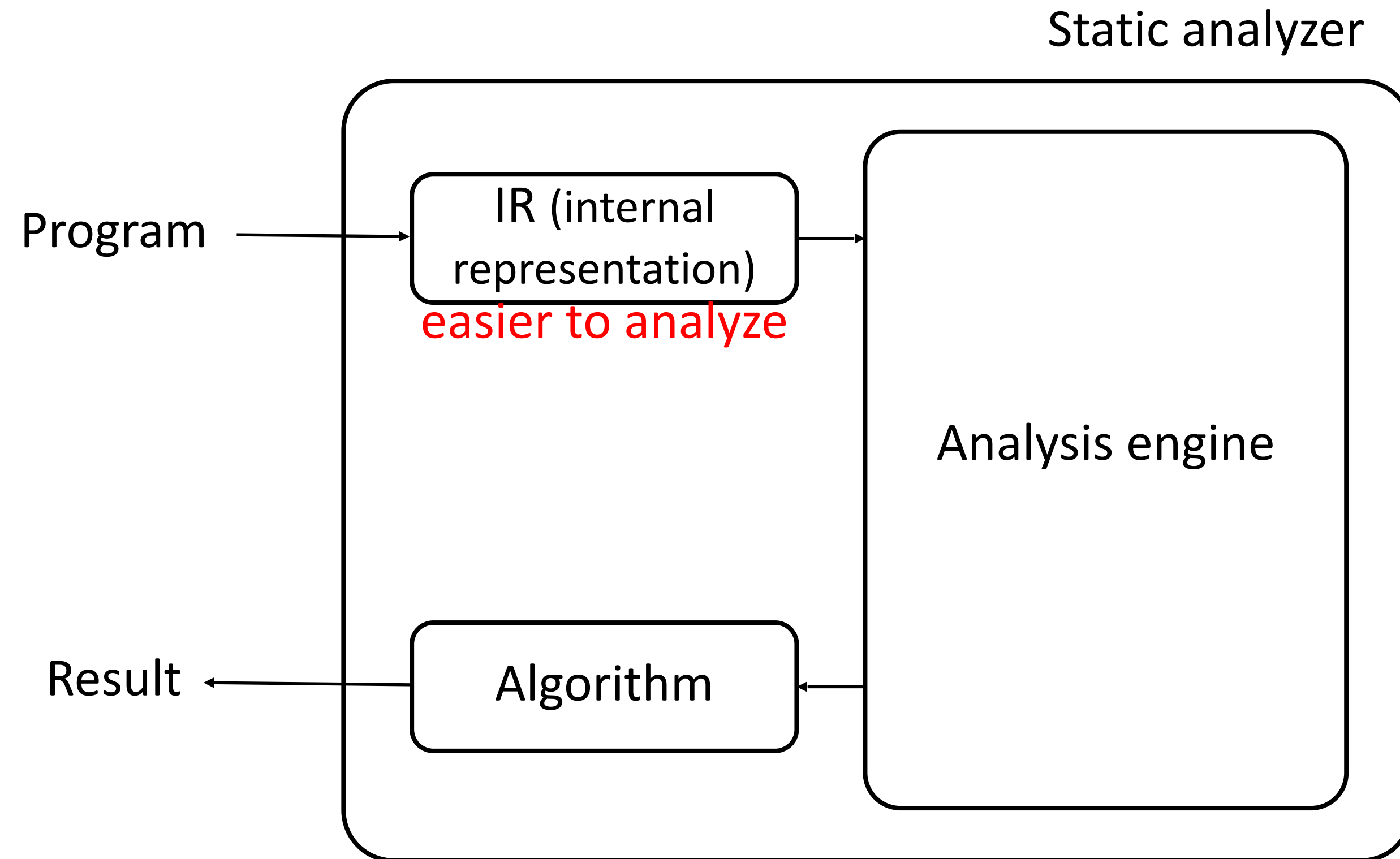


need to handle calls!
but what if we change domain?
we must handle this independently
from the domain we are using

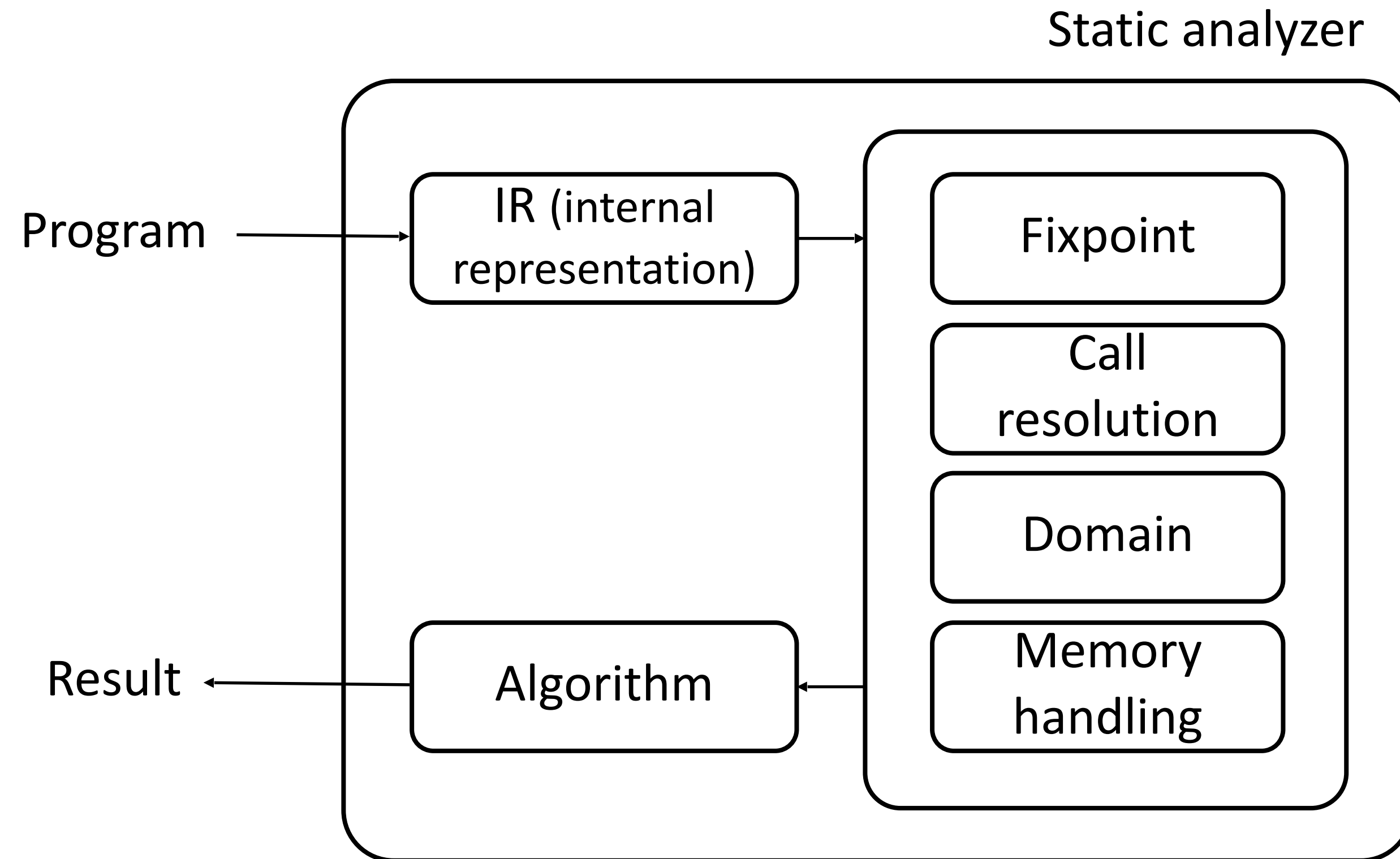
Static analyzer overview



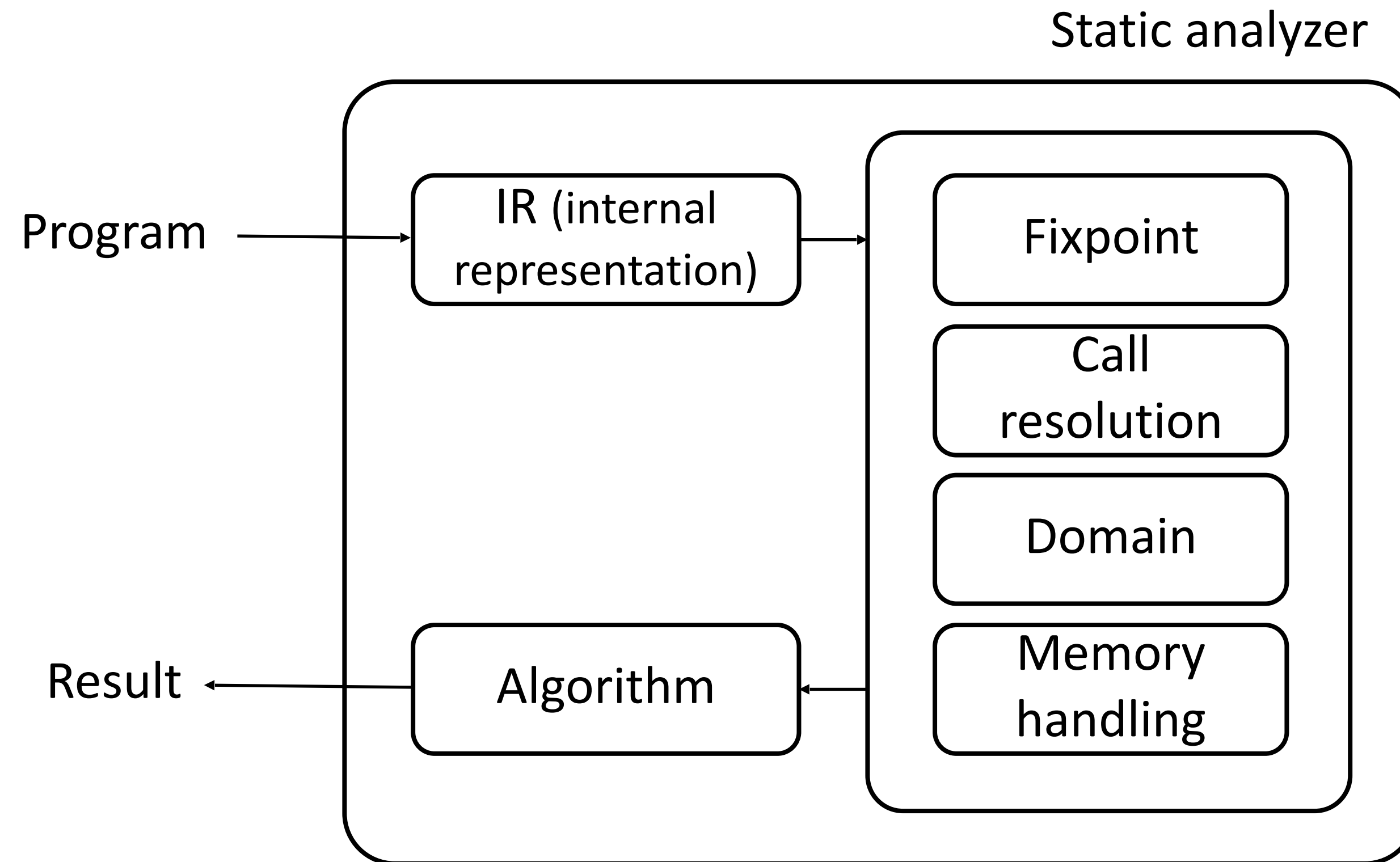
Static analyzer overview



Static analyzer overview

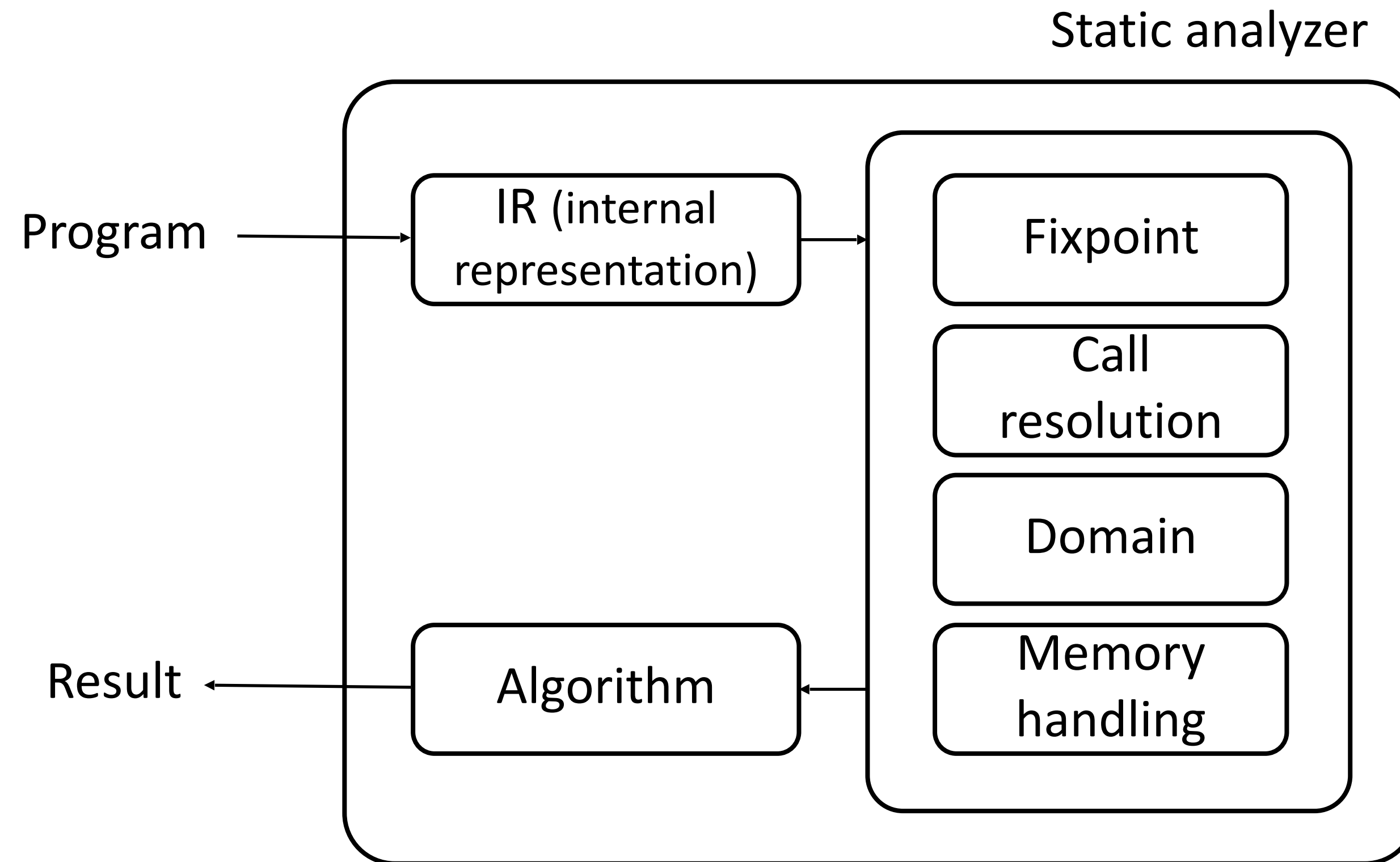


Static analyzer overview



Q: how to change analysis without rewriting everything?

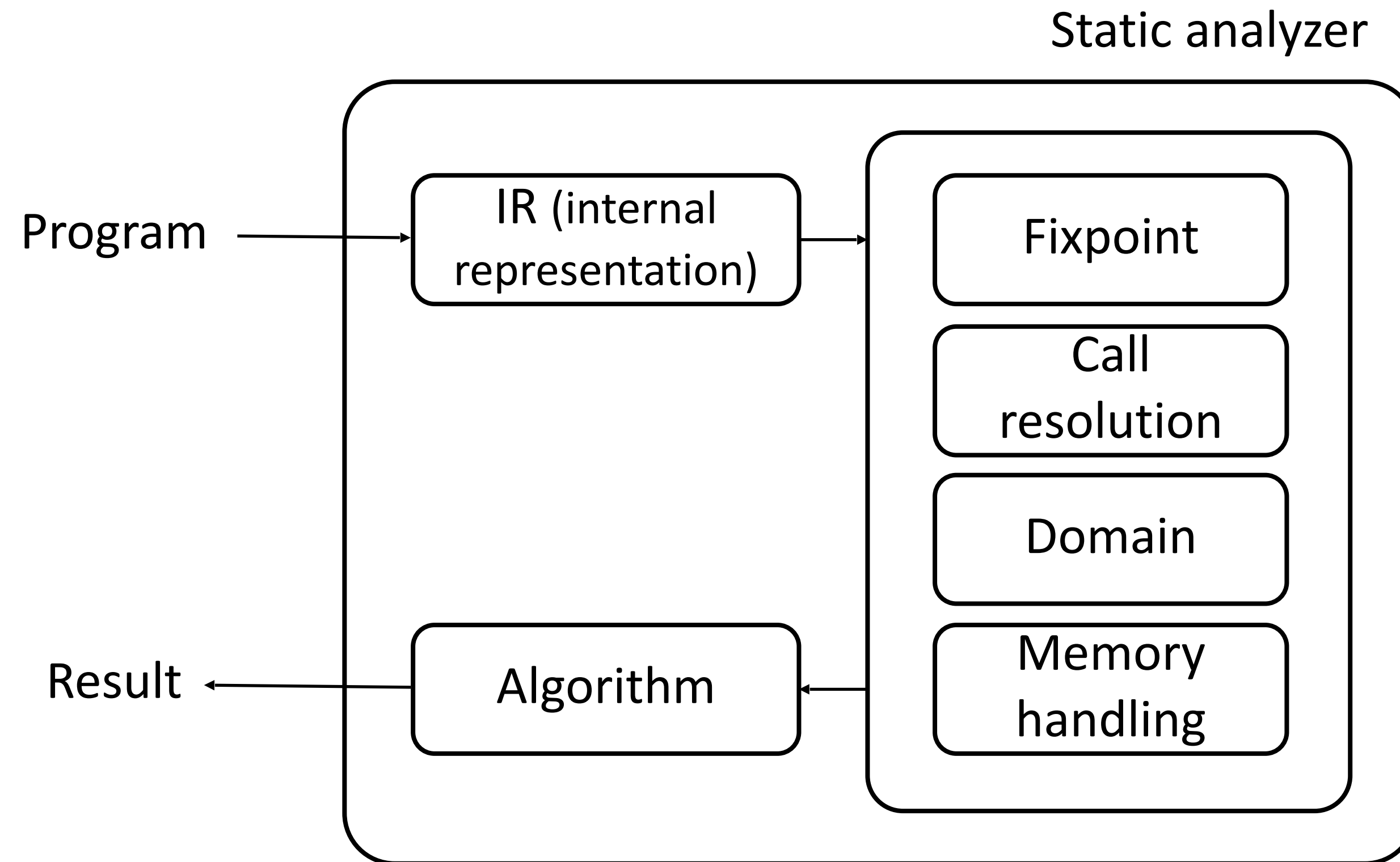
Static analyzer overview



Q: how to change analysis without rewriting everything?

A: analysis components have to be independent from each other

Static analyzer overview

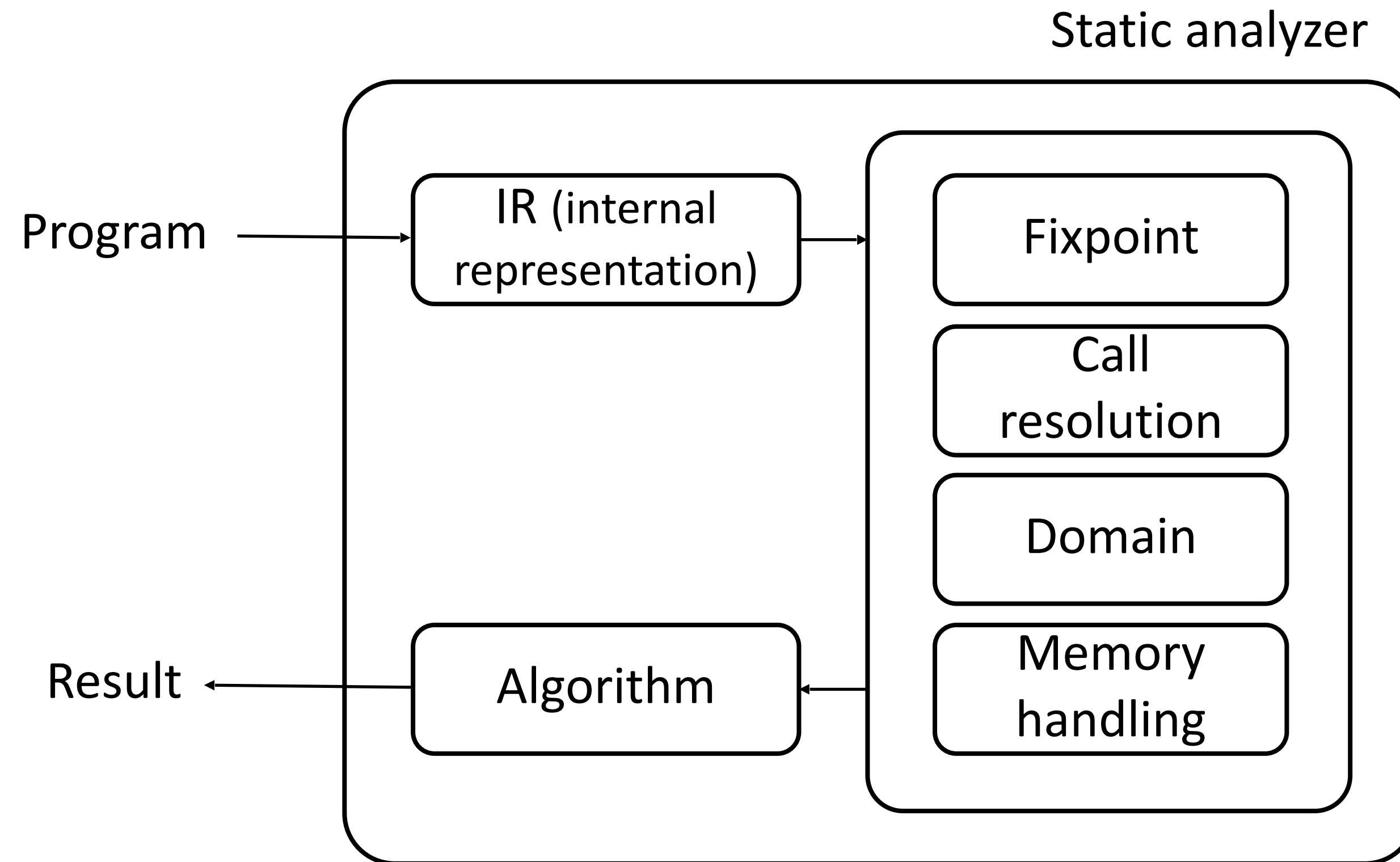


Q: how to change analysis without rewriting everything?

A: analysis components have to be independent from each other

Q: how to change language without rewriting everything?

Static analyzer overview



Q: how to change analysis without rewriting everything?

A: analysis components have to be independent from each other

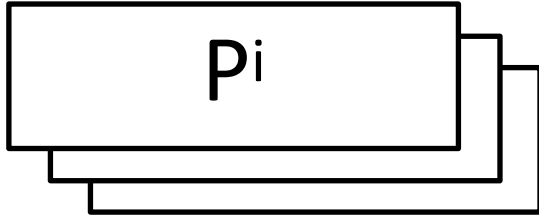
Q: how to change language without rewriting everything?

A: analysis components + IR should not be modeled after a specific language

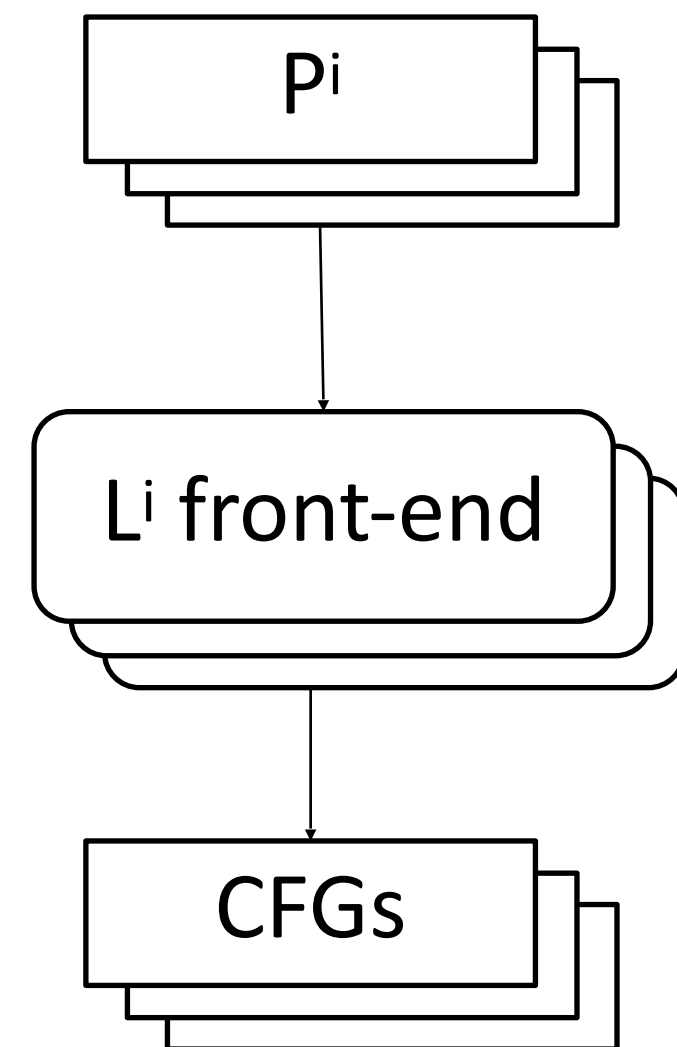
LiSA, a Library for Static Analysis

- Open source library written in Java
- Created the SSV group @ UniVE and maintained by UniPR and UniVE
- Library for creating static analyzers based on abstract interpretation
 - For multiple languages
 - For a variety of different domains

LiSA overview



LiSA overview

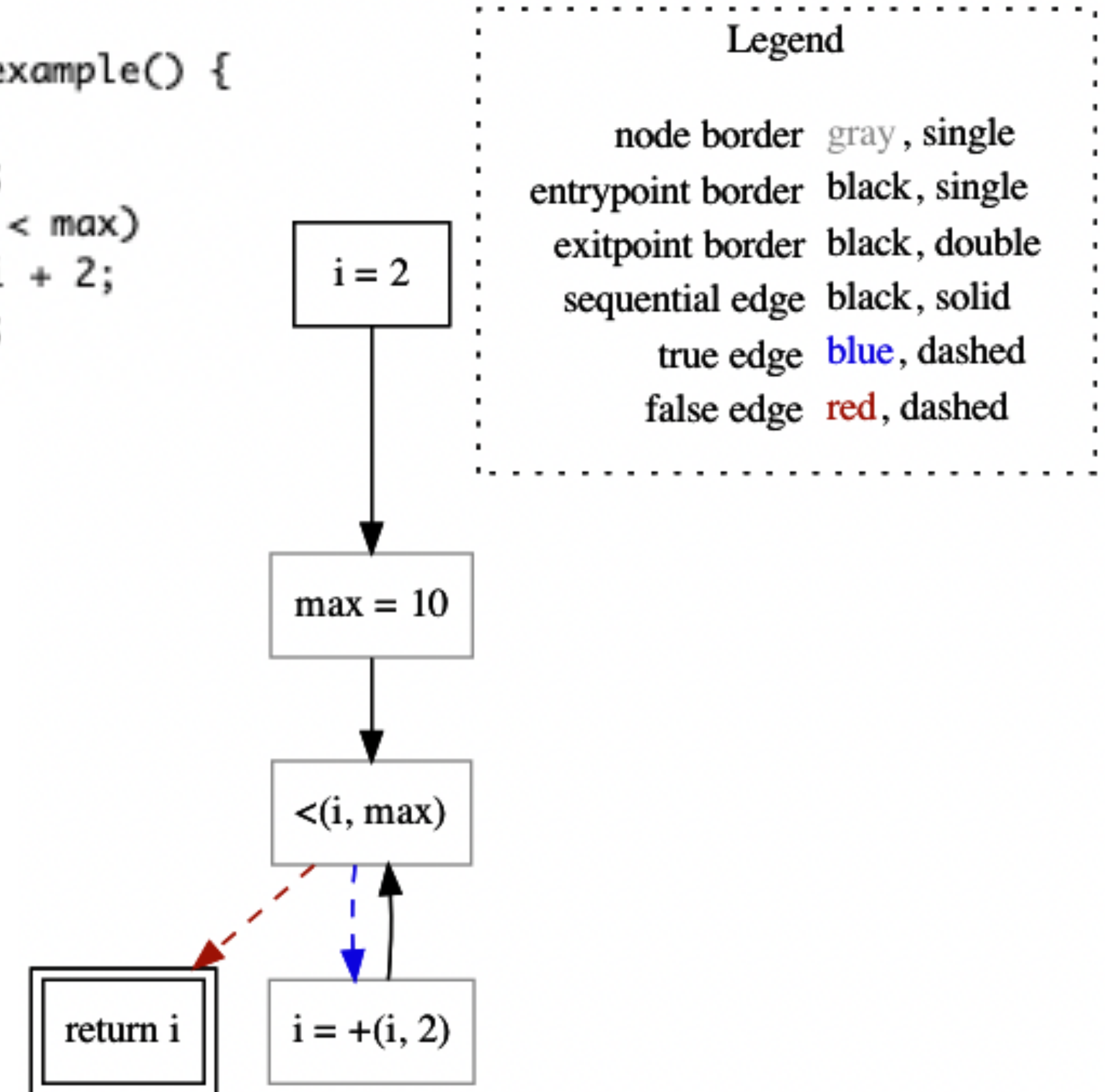


CFGs

A LiSA CFG is:

- Set of nodes (Statement)
- Set of edges linking nodes
 - SequentialEdge
 - TrueEdge (the stmt at source node is assumed to hold)
 - FalseEdge (the stmt at source node is assumed not to hold)

```
sign_parity_example() {  
    i = 2;  
    max = 10;  
    while (i < max)  
        i = i + 2;  
    return i;  
}
```



Why CFGs?

Why CFGs?

- Get rid of the syntax for control flow

Why CFGs?

- Get rid of the syntax for control flow
- Extensible: node instances are not fixed

Why CFGs?

- Get rid of the syntax for control flow
- Extensible: node instances are not fixed
- Received as inputs: no knowledge of source languages is kept past the frontends

Why CFGs?

- Get rid of the syntax for control flow
- Extensible: node instances are not fixed
- Received as inputs: no knowledge of source languages is kept past the frontends
- Easy to define fixpoints over graphs!

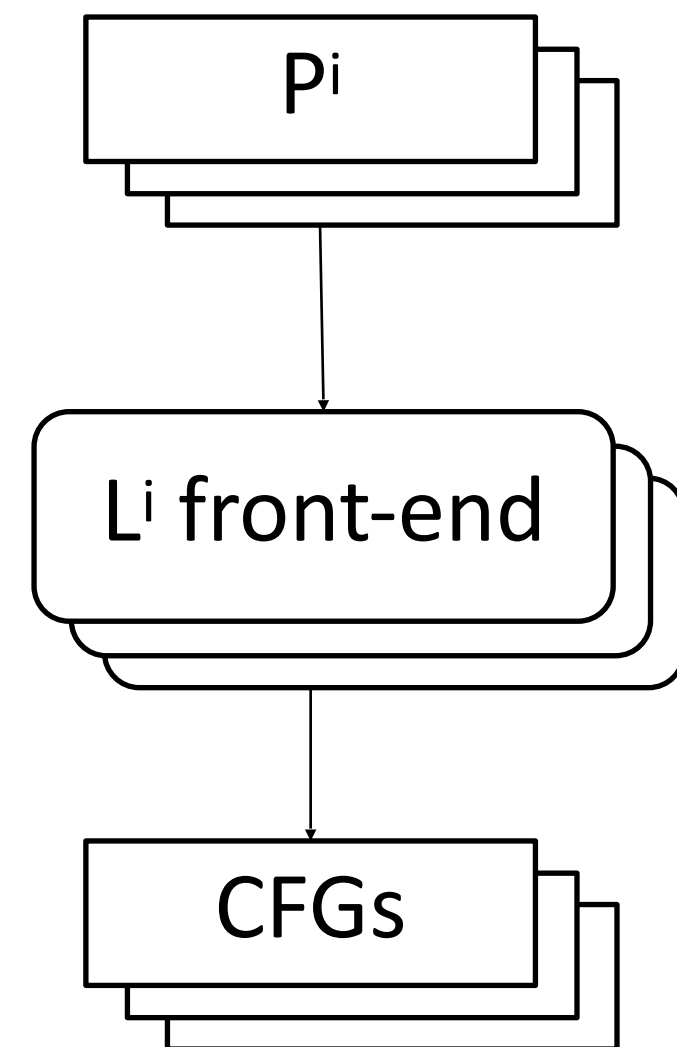
Why CFGs?

- Get rid of the syntax for control flow
- Extensible: node instances are not fixed
- Received as inputs: no knowledge of source languages is kept past the frontends
- Easy to define fixpoints over graphs!
- Can be produced in isolation: each frontend produces the CFGs for that language, and we can analyze them together

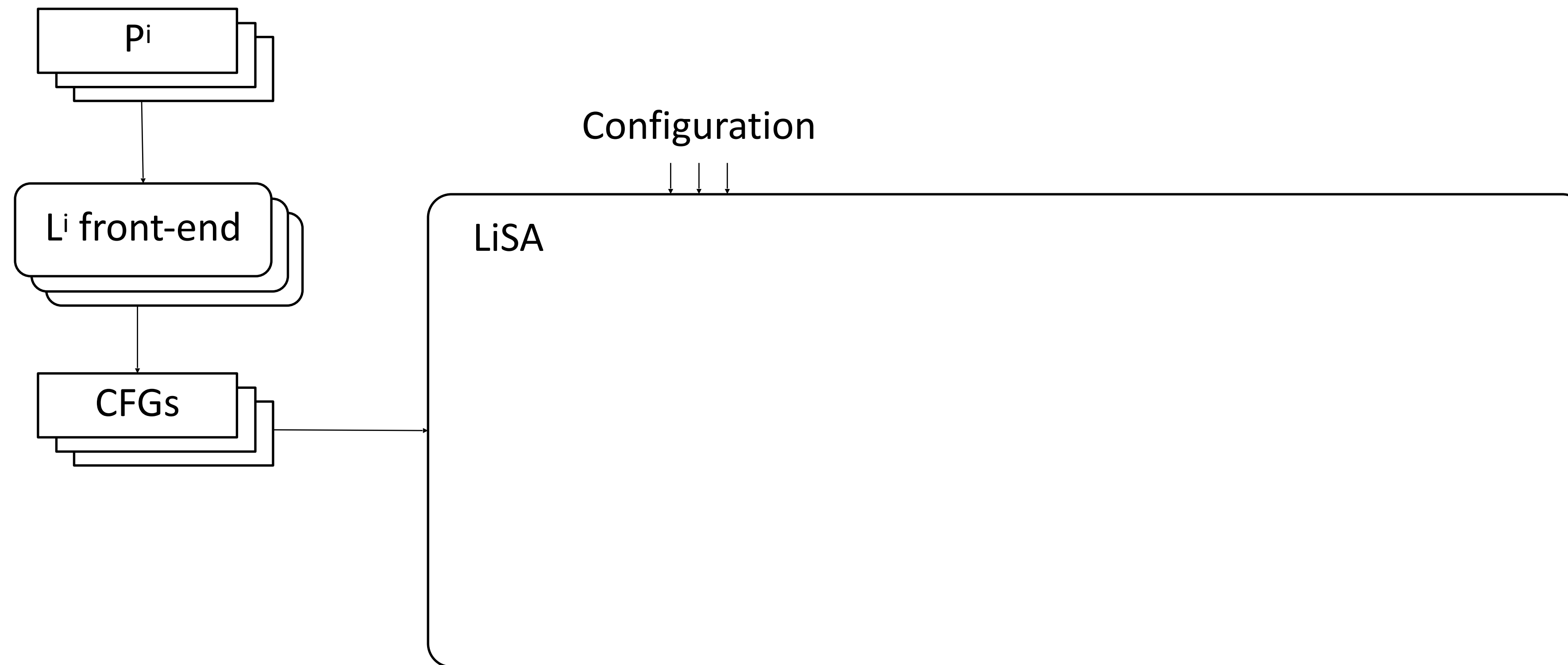
Why CFGs?

- Get rid of the syntax for control flow
- Extensible: node instances are not fixed
- Received as inputs: no knowledge of source languages is kept past the frontends
- Easy to define fixpoints over graphs!
- Can be produced in isolation: each frontend produces the CFGs for that language, and we can analyze them together
 - For the course, we will use a frontend for an IMP language (similar to Java)

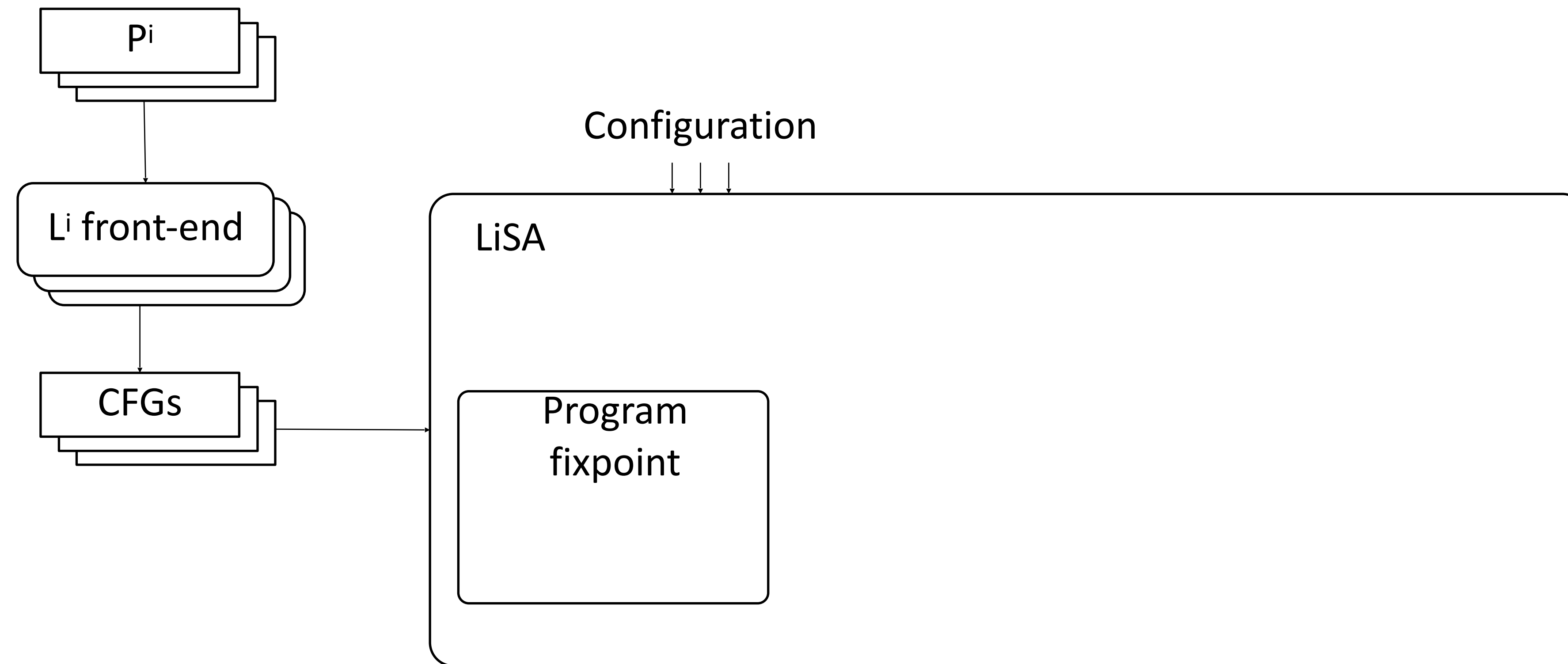
LiSA overview



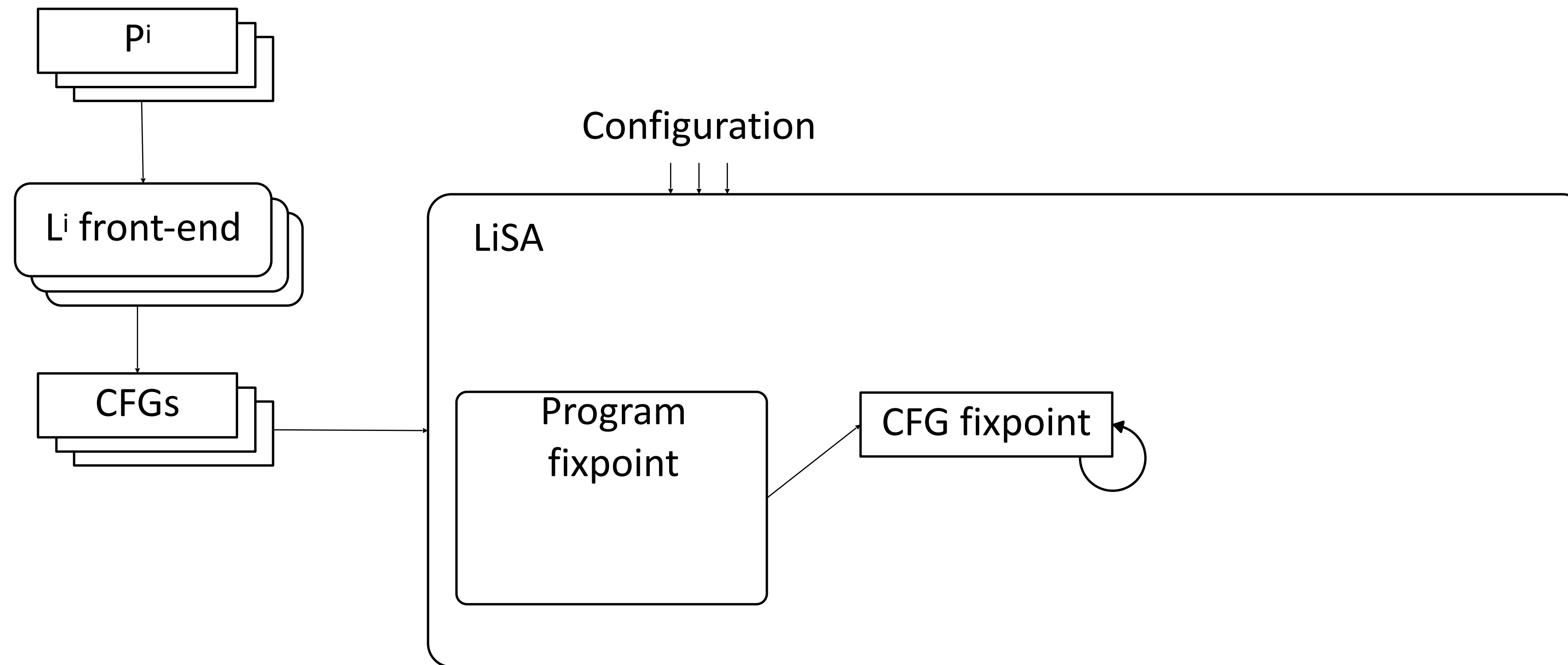
LiSA overview



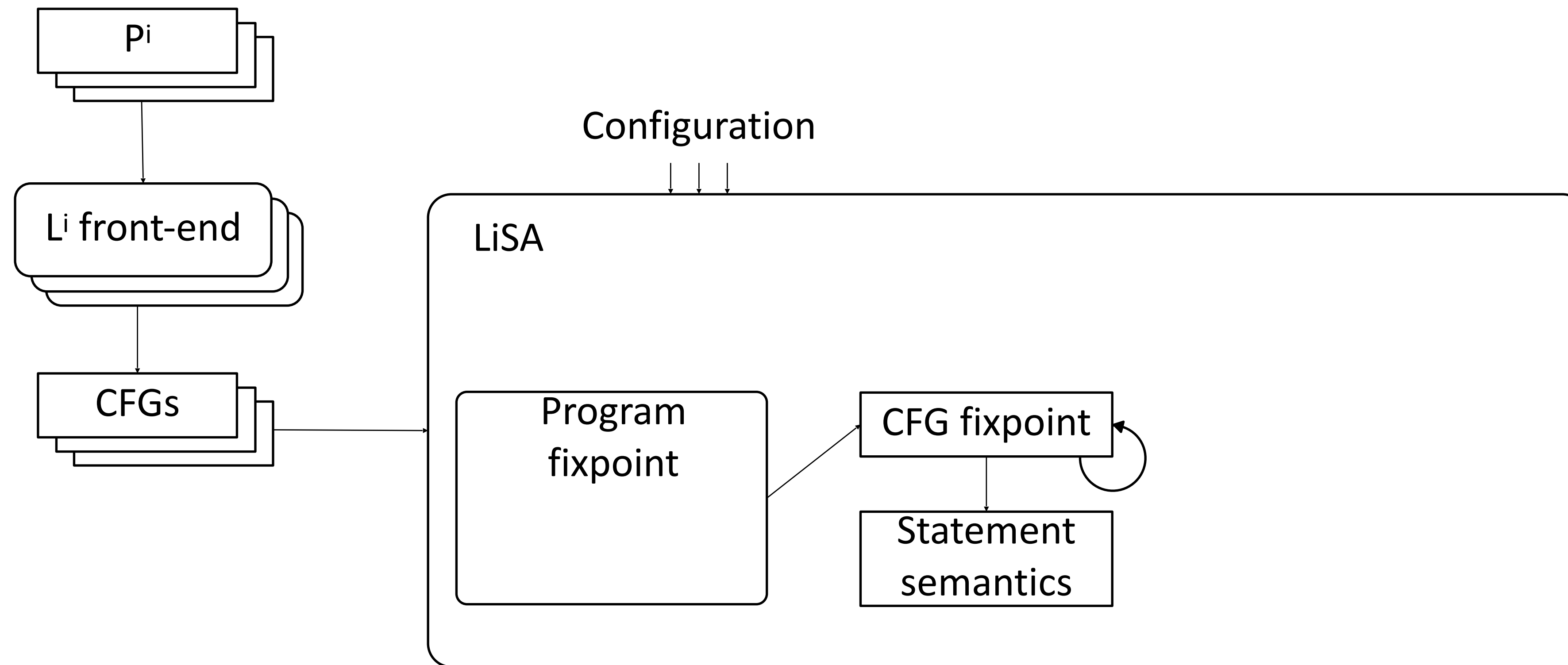
LiSA overview



LiSA overview



LiSA overview



Syntax vs semantics

Syntax vs semantics

- Statement (CFG nodes) express the syntax of the program

Syntax vs semantics

- Statement (CFG nodes) express the syntax of the program
- A statement can be complex: `x == null ? “unknown” : x.toString()`

Syntax vs semantics

- Statement (CFG nodes) express the syntax of the program
- A statement can be complex: `x == null ? "unknown" : x.toString()`
- Different statements can have the same meaning (semantics): `"a" + "b"` and `"a".concat("b")`

Syntax vs semantics

- Statement (CFG nodes) express the syntax of the program
- A statement can be complex: `x == null ? "unknown" : x.toString()`
- Different statements can have the same meaning (semantics): `"a" + "b"` and `"a".concat("b")`
- Can we simplify?

Syntax vs semantics

- Statement (CFG nodes) express the syntax of the program
- A statement can be complex: `x == null ? "unknown" : x.toString()`
- Different statements can have the same meaning (semantics): `"a" + "b"` and `"a".concat("b")`
- Can we simplify?
 - Yes! The `SymbolicExpressions` language

Syntax vs semantics

- Statement (CFG nodes) express the syntax of the program
- A statement can be complex: `x == null ? "unknown" : x.toString()`
- Different statements can have the same meaning (semantics): `"a" + "b"` and `"a".concat("b")`
- Can we simplify?
 - Yes! The `SymbolicExpressions` language
 - Domains work on an internal language of `SymbolicExpressions`

Syntax vs semantics

- Statement (CFG nodes) express the syntax of the program
- A statement can be complex: `x == null ? "unknown" : x.toString()`
- Different statements can have the same meaning (semantics): `"a" + "b"` and `"a".concat("b")`
- Can we simplify?
 - Yes! The `SymbolicExpressions` language
 - Domains work on an internal language of `SymbolicExpressions`
 - Each symbolic expression is an “atomic” semantic operation

Syntax vs semantics

- Statement (CFG nodes) express the syntax of the program
- A statement can be complex: `x == null ? "unknown" : x.toString()`
- Different statements can have the same meaning (semantics): `"a" + "b"` and `"a".concat("b")`
- Can we simplify?
 - Yes! The `SymbolicExpressions` language
 - Domains work on an internal language of `SymbolicExpressions`
 - Each symbolic expression is an “atomic” semantic operation
 - `concatenation`, `addition`, ...

Syntax vs semantics

- Statement (CFG nodes) express the syntax of the program
- A statement can be complex: `x == null ? "unknown" : x.toString()`
- Different statements can have the same meaning (semantics): `"a" + "b"` and `"a".concat("b")`
- Can we simplify?
 - Yes! The `SymbolicExpressions` language
 - Domains work on an internal language of `SymbolicExpressions`
 - Each symbolic expression is an “atomic” semantic operation
 - `concatenation`, `addition`, ...
 - Fixpoint uses `Statement.semantics()` to rewrite each `Statement` to `SymbolicExpression(s)` and feed them to the domain

Examples

Pseudocode

Addition.semantics():

```
return domain.eval(new ArithmSum(left, right))
```

Concatenation.semantics():

```
return domain.eval(new StringConcat(left, right))
```

Plus.semantics():

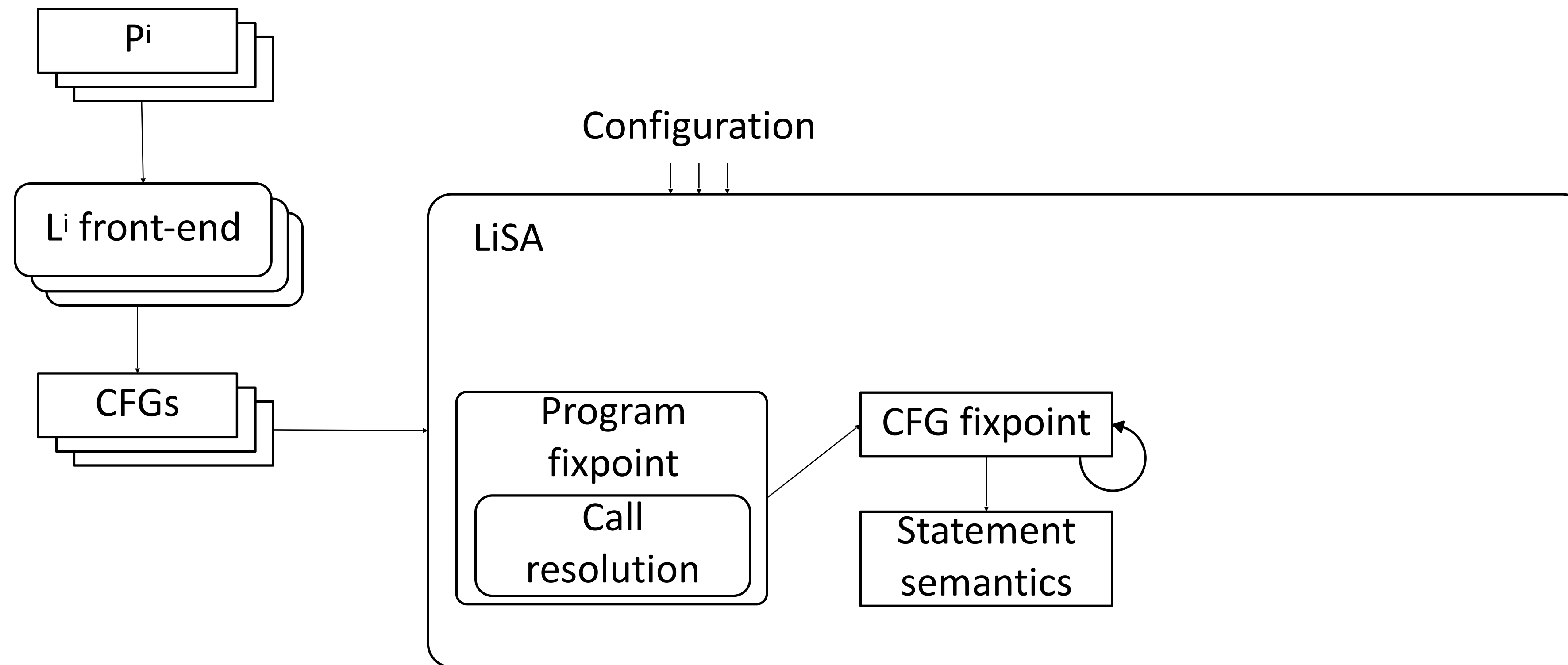
```
if left is string || right is string:
```

```
    return domain.eval(new StringConcat(left, right))
```

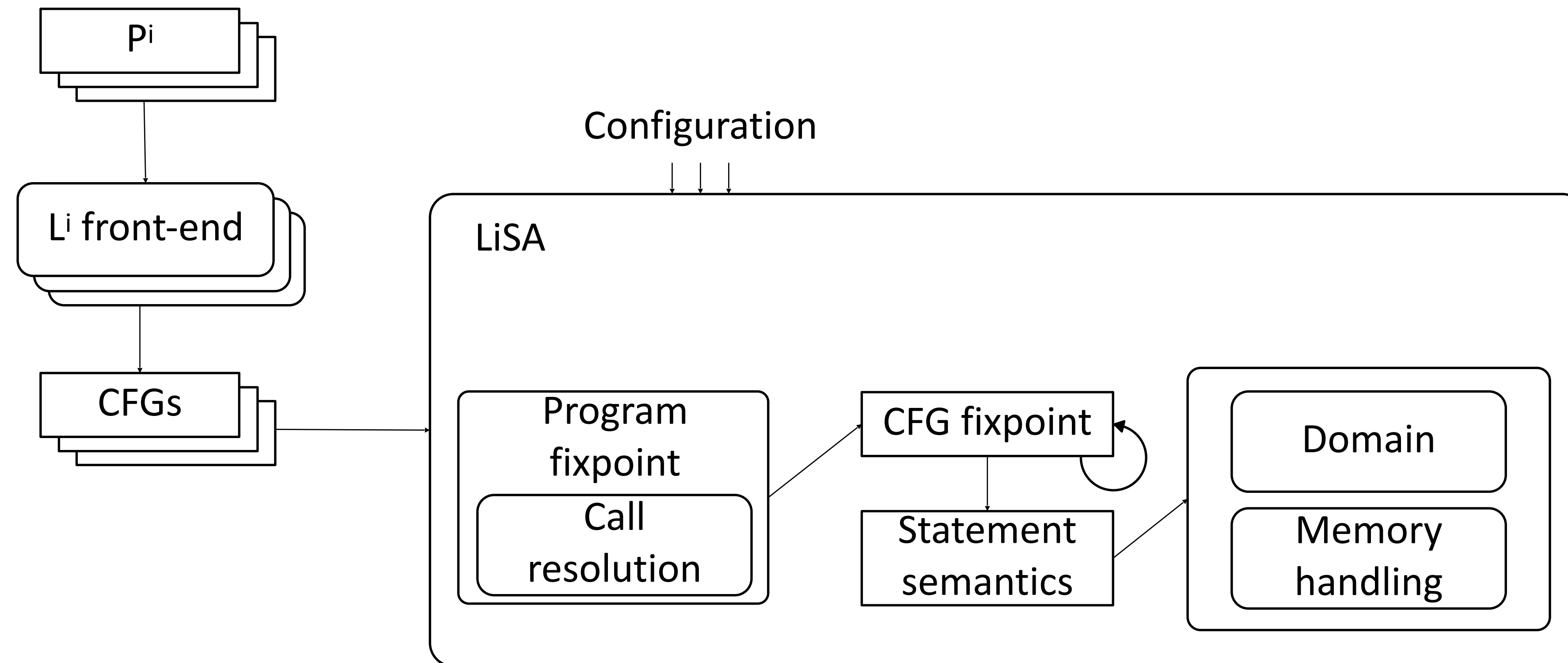
```
else:
```

```
    return domain.eval(new ArithmSum(left, right))
```

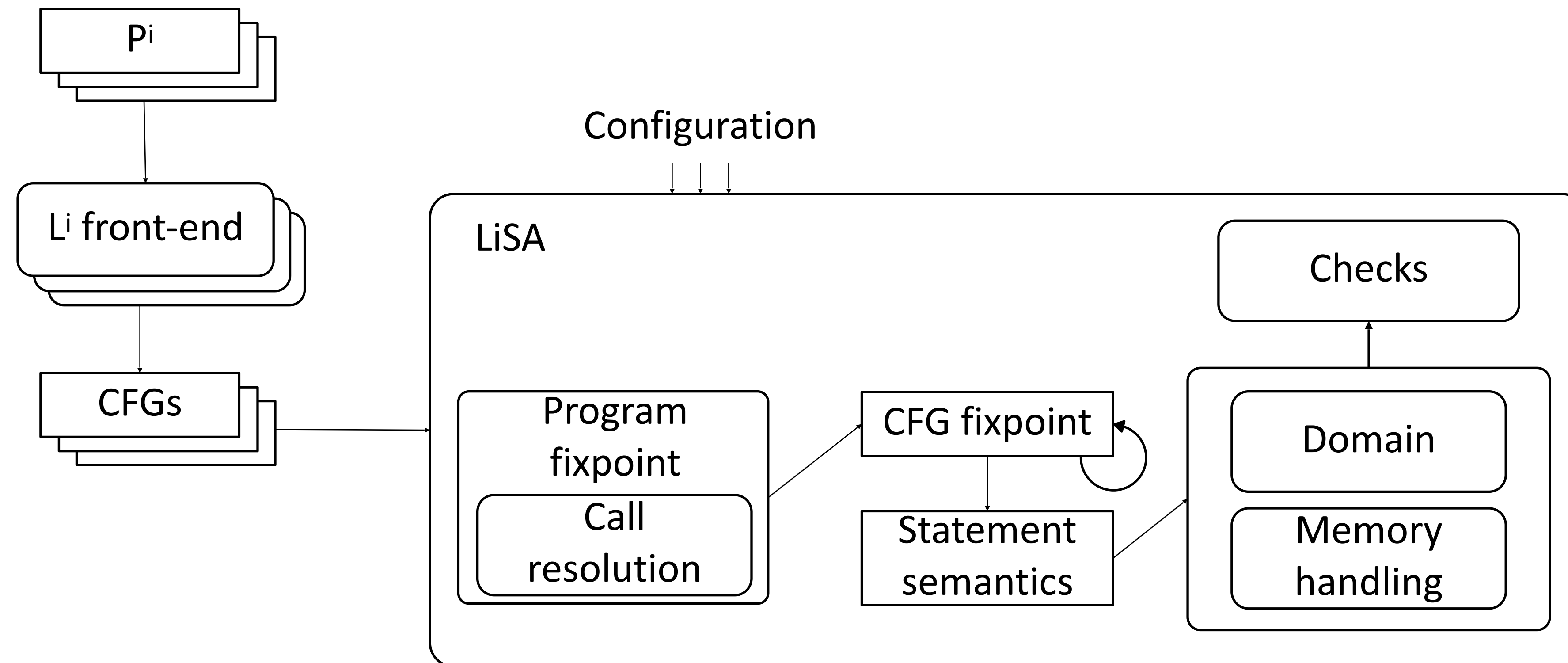
LiSA overview



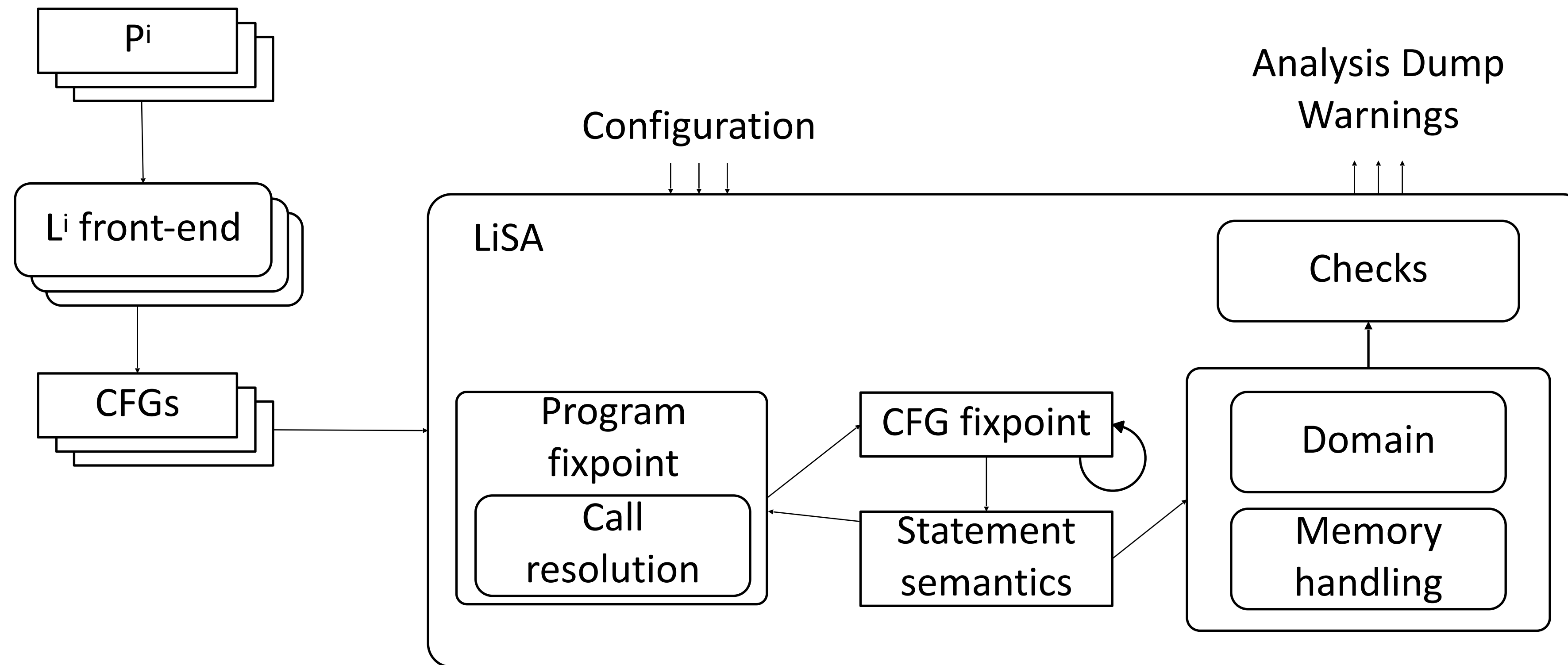
LiSA overview



LiSA overview

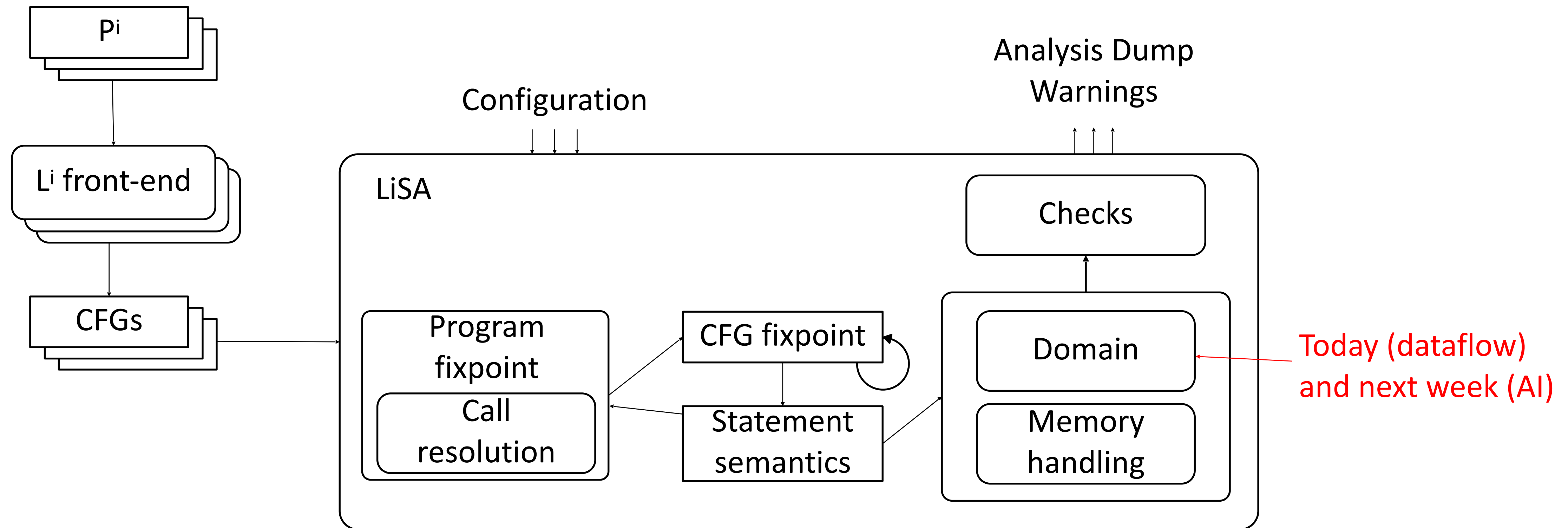


LiSA overview



Today (dataflow)
and next week (AI)

LiSA overview



Questions?

Recall: Dataflow

“Data-flow analysis is a technique for gathering information about the possible set of values calculated at various points in a computer program” – Wikipedia

- Forward or Backward
- Possible or Definite

Key aims:

- Modularity: factor out common components
- As close as possible to the formalization

Recall: Dataflow

“Data-flow analysis is a technique for gathering information about the possible set of values calculated at various points in a computer program” – Wikipedia

- Forward or ~~Backward~~ **Not supported in LiSA**
- Possible or Definite

Key aims:

- Modularity: factor out common components
- As close as possible to the formalization

Forward dataflow analysis

```
 $f_n(v) = v \setminus kill_n \cup gen_n$  _____  
for all  $n \in N$  do  
     $out_n \leftarrow f_n(\perp)$   
end for  
 $in_{n_0} \leftarrow I$   
 $out_{n_0} \leftarrow f_{n_0}(I)$   
 $ws \leftarrow N \setminus \{n_0\}$   
while  $ws \neq \emptyset$  do  
     $n \leftarrow pop(ws)$   
     $in_n \leftarrow \oplus \{out_m : m \in preds(n)\}$   
     $out_n \leftarrow f_n(in_n)$   
    if  $out_n$  changed then  
         $ws \leftarrow ws \cup succ(n)$   
    end if  
end while
```

Forward dataflow analysis

$f_n(v) = v \setminus \boxed{kill_n \cup gen_n}$ _____ Domain-specific

for all $n \in N$ do
 $out_n \leftarrow f_n(\perp)$
end for
 $in_{n_0} \leftarrow I$
 $out_{n_0} \leftarrow f_{n_0}(I)$
 $ws \leftarrow N \setminus \{n_0\}$
while $ws \neq \emptyset$ do
 $n \leftarrow pop(ws)$
 $in_n \leftarrow \oplus \{out_m : m \in preds(n)\}$
 $out_n \leftarrow f_n(in_n)$
 if out_n changed then
 $ws \leftarrow ws \cup succ(n)$
 end if
end while

Forward dataflow analysis

```
 $f_n(v) = v \setminus kill_n \cup gen_n$   
for all  $n \in N$  do  
   $out_n \leftarrow f_n(\perp)$   
end for  
 $in_{n_0} \leftarrow I$   
 $out_{n_0} \leftarrow f_{n_0}(I)$   
 $ws \leftarrow N \setminus \{n_0\}$   
while  $ws \neq \emptyset$  do  
   $n \leftarrow pop(ws)$   
   $in_n \leftarrow \oplus \{out_m : m \in preds(n)\}$   
   $out_n \leftarrow f_n(in_n)$   
  if  $out_n$  changed then  
     $ws \leftarrow ws \cup succ(n)$   
  end if  
end while
```

Domain-specific

No information (definite: empty set, possible: all elements)

Forward dataflow analysis

```
 $f_n(v) = v \setminus \boxed{kill_n \cup gen_n}$   
for all  $n \in N$  do  
   $out_n \leftarrow f_n(\boxed{\perp})$   
end for  
 $in_{n_0} \leftarrow \boxed{I}$   
 $out_{n_0} \leftarrow f_{n_0}(\boxed{I})$   
 $ws \leftarrow N \setminus \{n_0\}$   
while  $ws \neq \emptyset$  do  
   $n \leftarrow pop(ws)$   
   $in_n \leftarrow \oplus \{out_m : m \in preds(n)\}$   
   $out_n \leftarrow f_n(in_n)$   
  if  $out_n$  changed then  
     $ws \leftarrow ws \cup succ(n)$   
  end if  
end while
```

Domain-specific

No information (definite: empty set, possible: all elements)

Initial value (definite: empty set, possible: all elements)

Forward dataflow analysis

```
 $f_n(v) = v \setminus \boxed{kill_n \cup gen_n}$   
for all  $n \in N$  do  
   $out_n \leftarrow f_n(\boxed{\perp})$   
end for  
 $in_{n_0} \leftarrow \boxed{I}$   
 $out_{n_0} \leftarrow f_{n_0}(\boxed{I})$   
 $ws \leftarrow N \setminus \{n_0\}$   
while  $ws \neq \emptyset$  do  
   $n \leftarrow pop(ws)$   
   $in_n \leftarrow \boxed{\oplus} \{out_m : m \in preds(n)\}$   
   $out_n \leftarrow f_n(in_n)$   
  if  $out_n$  changed then  
     $ws \leftarrow ws \cup succ(n)$   
  end if  
end while
```

Domain-specific

No information (definite: empty set, possible: all elements)

Initial value (definite: empty set, possible: all elements)

Join operation (definite: intersection, possible: union)

Forward dataflow analysis

$f_n(v) = v \setminus \boxed{kill_n \cup gen_n}$ → Domain-specific

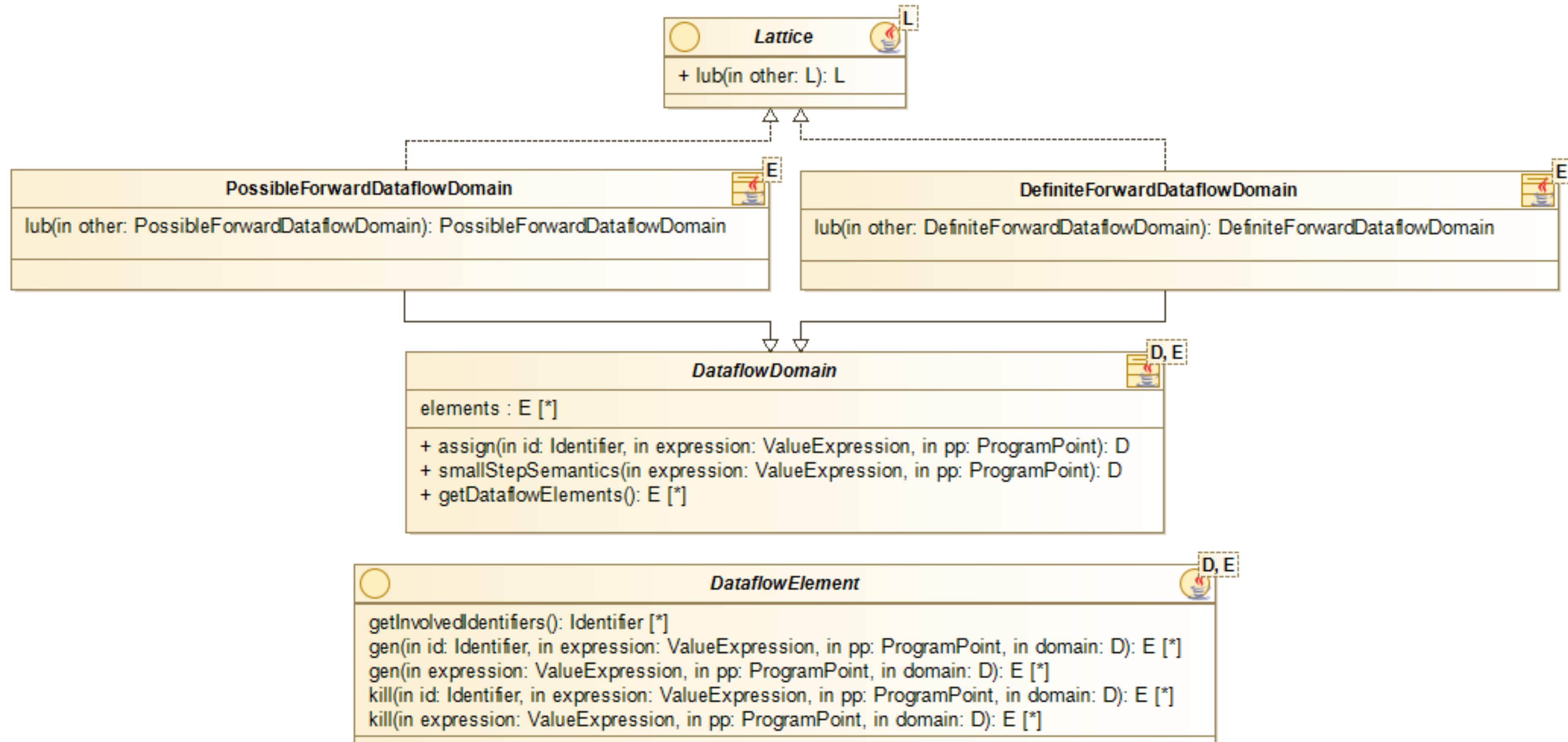
for all $n \in N$ do
 $out_n \leftarrow f_n(\boxed{\perp})$ → No information (definite: empty set, possible: all elements)
end for
 $in_{n_0} \leftarrow \boxed{I}$ → Initial value (definite: empty set, possible: all elements)
 $out_{n_0} \leftarrow f_{n_0}(\boxed{I})$ → Initial value (definite: empty set, possible: all elements)
 $ws \leftarrow N \setminus \{n_0\}$
while $ws \neq \emptyset$ do
 $n \leftarrow pop(ws)$
 $in_n \leftarrow \boxed{\oplus \{out_m : m \in preds(n)\}}$ → Join operation (definite: intersection, possible: union)
 $out_n \leftarrow f_n(in_n)$
 if out_n changed then
 $ws \leftarrow ws \cup succ(n)$
 end if
end while

How can we modularize?

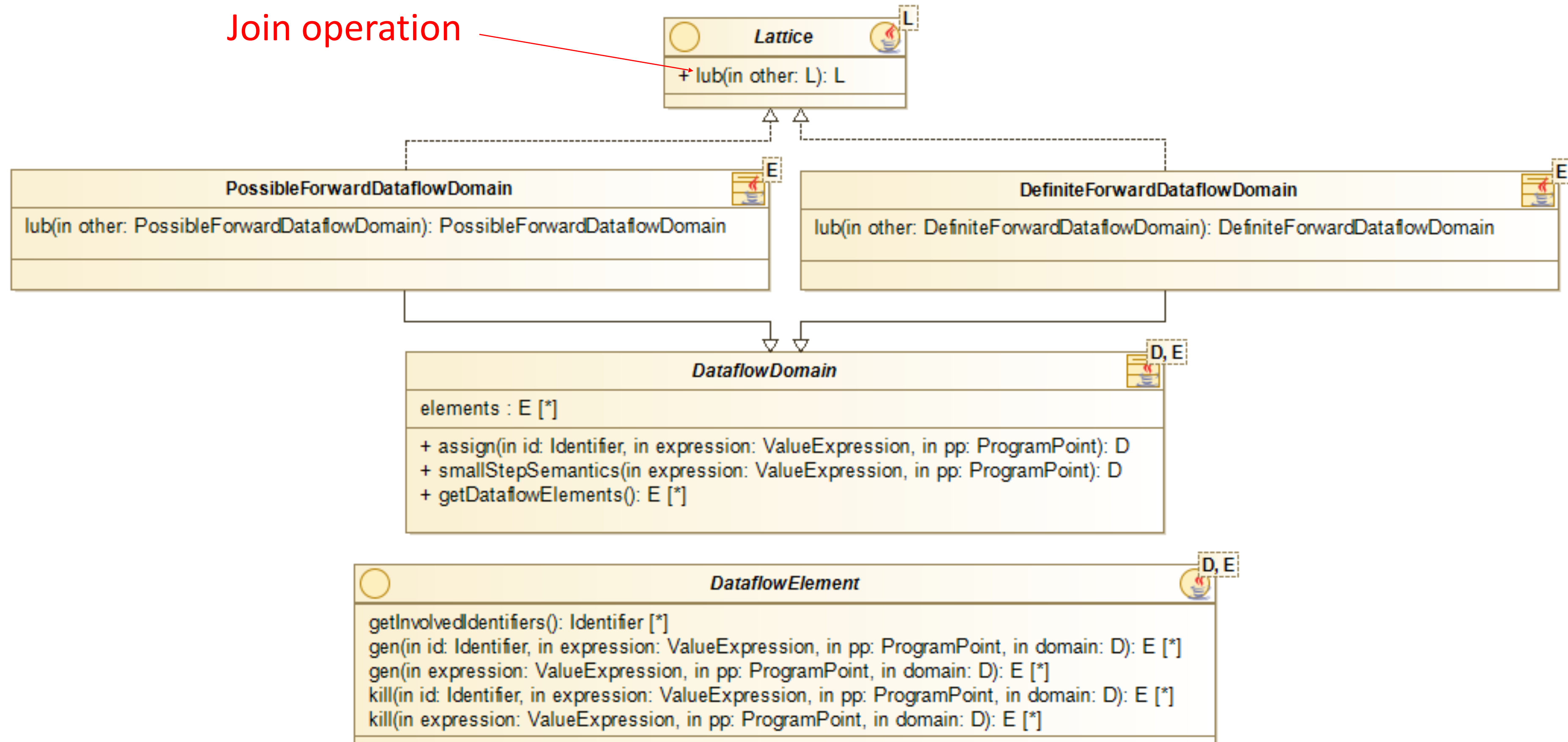
Dataflow in LiSA

- Dataflow domains are a kind of Domain
- Worklist algorithm is CFG's fixpoint
 - Unique!
- Factor common logic in DataflowDomain:
 - Assignment and expression evaluation logic are defined here
 - DefiniteDomain: join is intersection
 - PossibleDomain: join is union
 - No need to re-implement the logic
- Implementation specific DataflowElement define gen/kill
 - One domain $\rightarrow$ one instance of this class

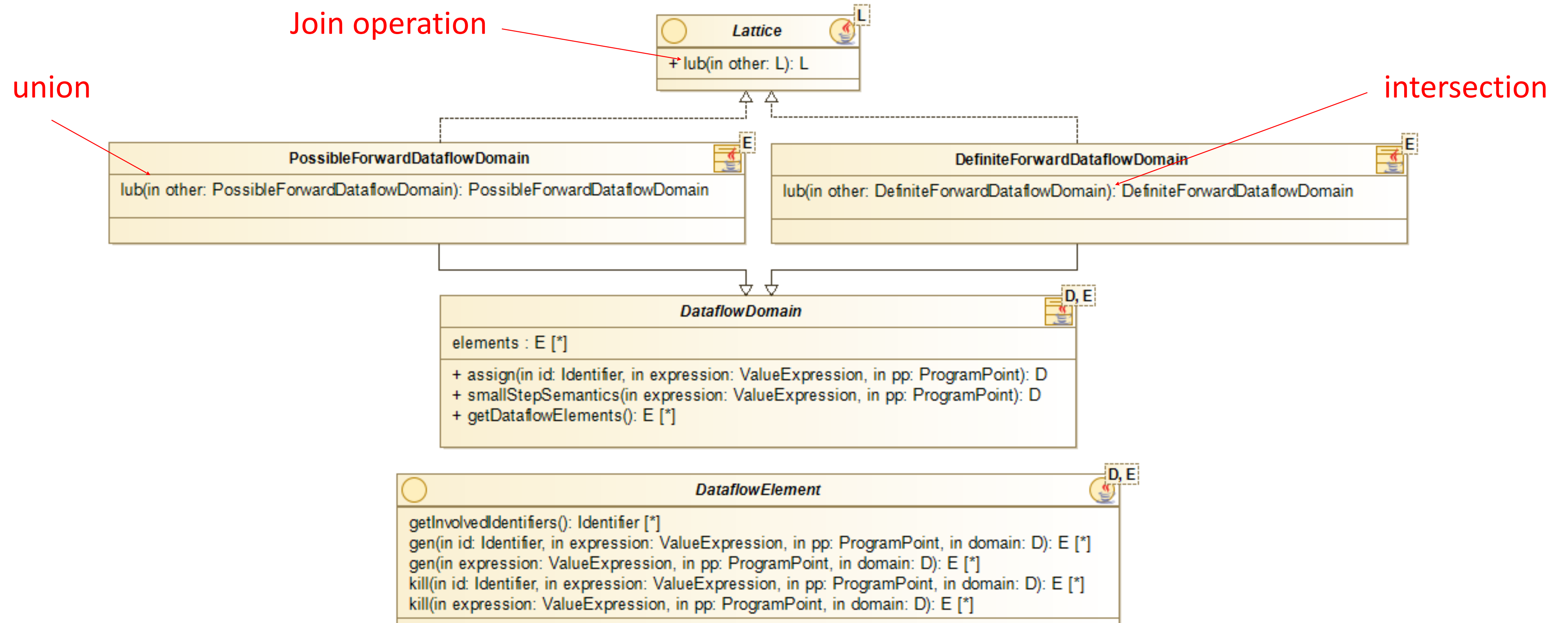
Dataflow classes



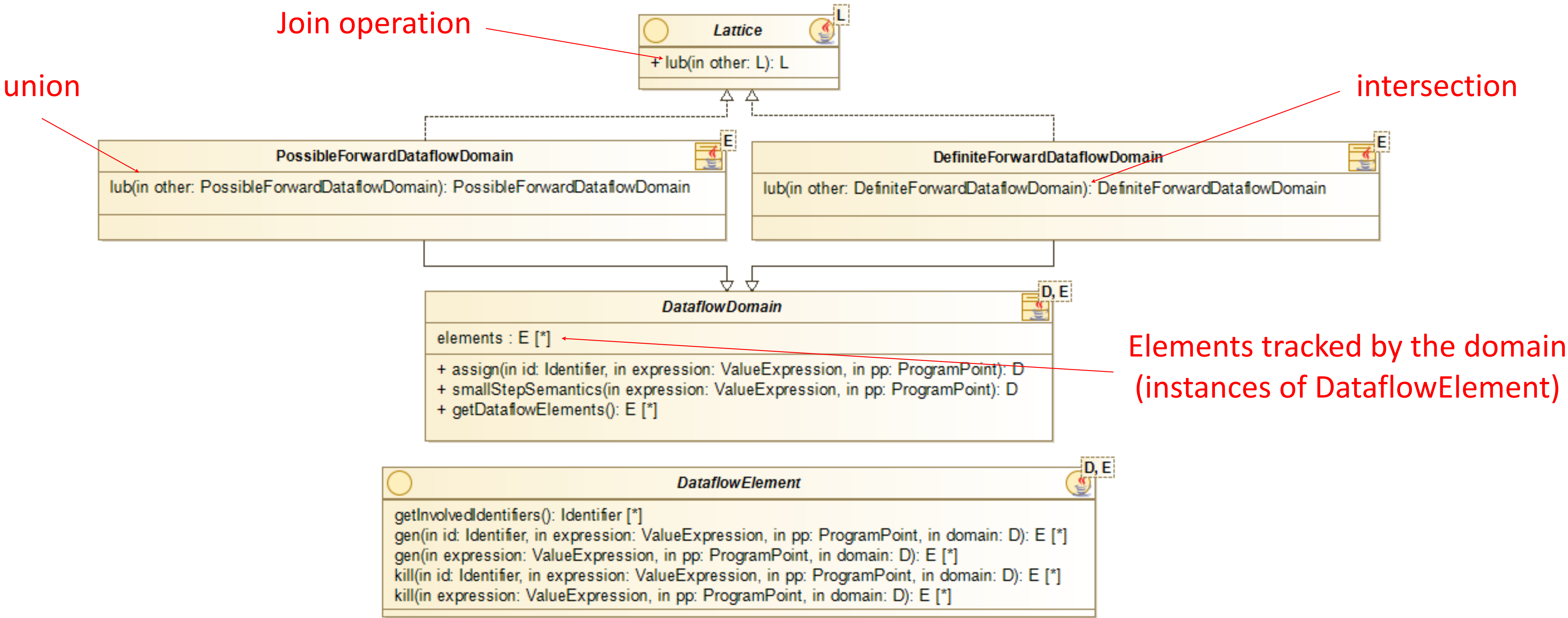
Dataflow classes



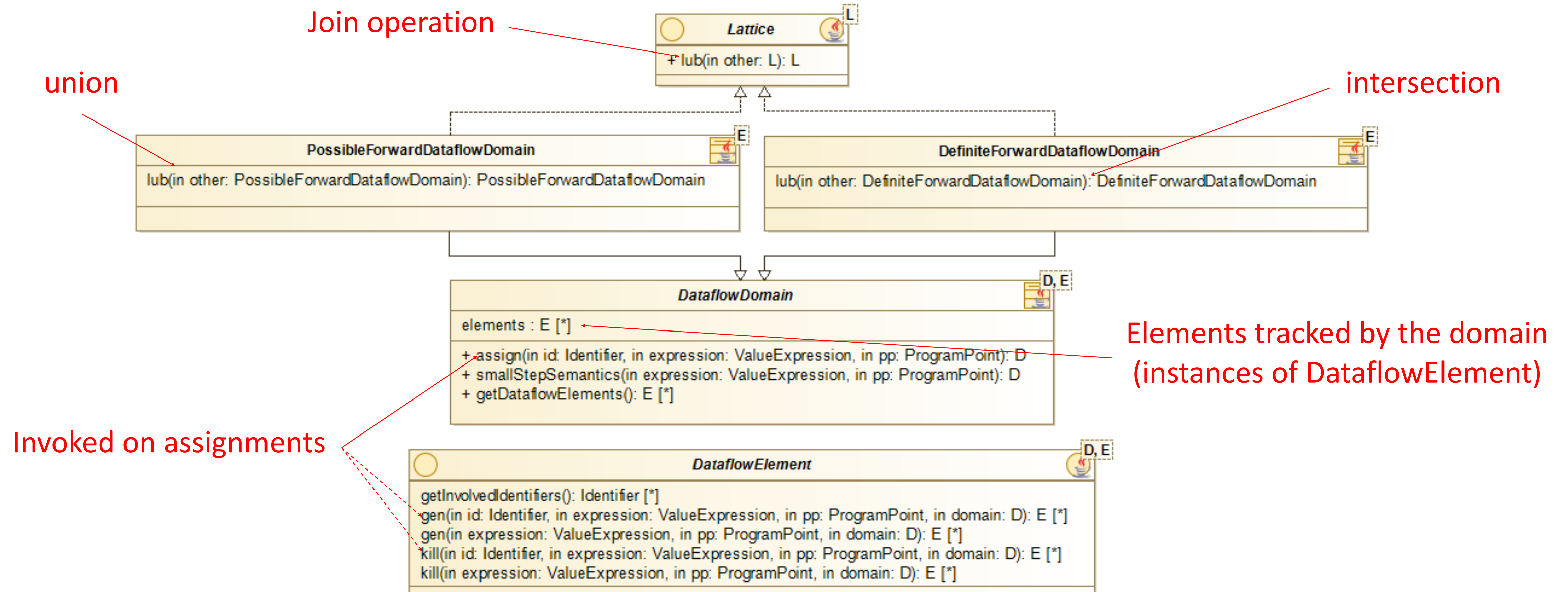
Dataflow classes



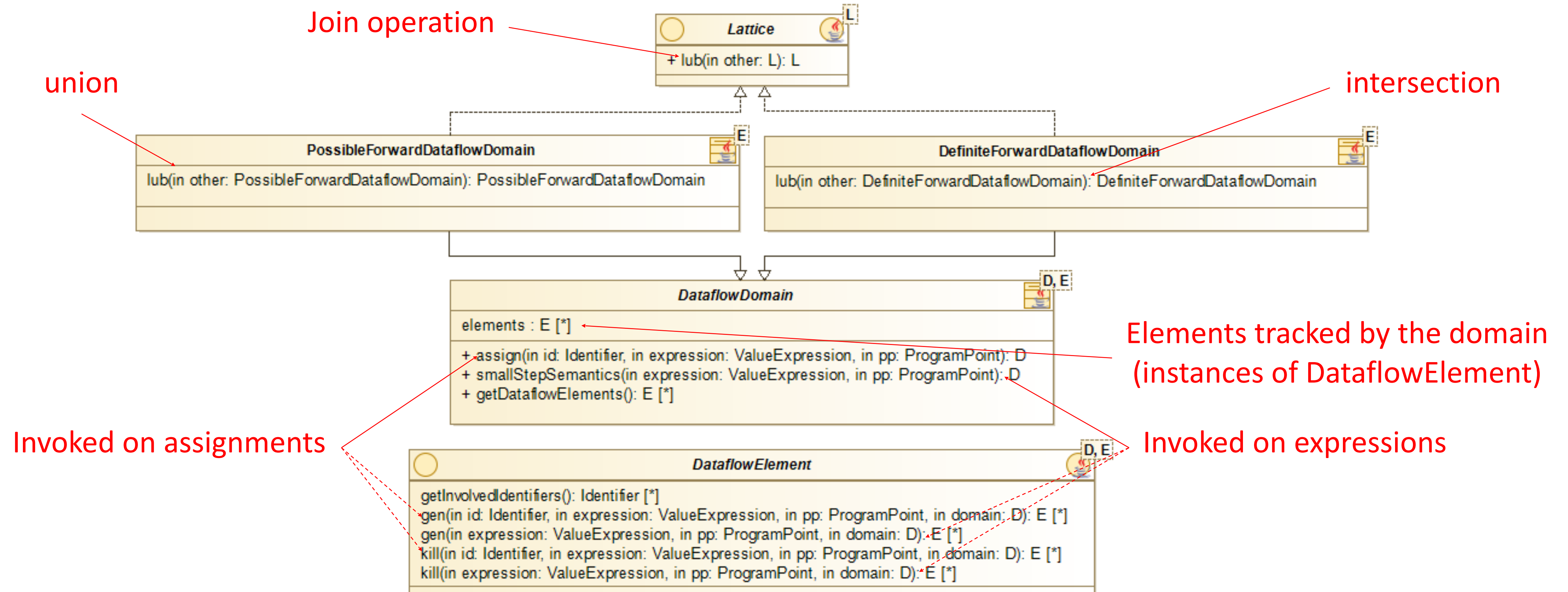
Dataflow classes



Dataflow classes



Dataflow classes



Available expression

Coding demo

Syntactic checkers

Coding demo

Reaching definition

Exercise